

# Apache OpenOffice

Version 4.1

## Calc Guide

AOO Documentation Team

### **Appendix B**

### **Description of Functions**

# Copyright

This document is Copyright © 2025 by The Apache Software Foundation. You may distribute it and/or modify it under the terms of the Creative Commons Attribution License, version 3.0 or later (<https://creativecommons.org/licenses/by/3.0/>).

Apache, Apache OpenOffice, and OpenOffice.org are trademarks of the Apache Software Foundation. Used with permission.

# Acknowledgments

This document is updated from a previous version by the OOoAuthors team. The AOO Documentation Team appreciates the many contributions of Michael Scarborough and Michele Petrovsky.

# Publication date and software version

Published August 25, 2025. Based on Apache OpenOffice 4.1.

# Note for Mac Users

Some keystrokes and menu items differ on a Mac from those used in Windows and Linux. The table below gives some common substitutions for the instructions in this chapter. For a more detailed list, see the application Help.

| <i><b>Windows/Linux</b></i>                 | <i><b>Mac equivalent</b></i>       | <i><b>Effect</b></i>            |
|---------------------------------------------|------------------------------------|---------------------------------|
| <b>Tools &gt; Options</b><br>menu selection | <b>OpenOffice &gt; Preferences</b> | Access setup options            |
| <i>Right-click</i>                          | <i>Control+click</i>               | Open context menu               |
| <i>Ctrl</i> (Control)                       | ⌘ (Command)                        | Used with other keys            |
| <i>F5</i>                                   | <i>Shift+⌘+F5</i>                  | Open the Navigator              |
| <i>F11</i>                                  | <i>⌘+T</i>                         | Open Styles & Formatting window |

# Contents

---

## Appendix B

|                                                |          |
|------------------------------------------------|----------|
| <b>Description of Functions</b>                | <b>1</b> |
| Functions Available in Calc                    | 4        |
| Terminology: Numbers and Arguments             | 4        |
| Mathematical Functions                         | 4        |
| Financial Analysis Functions                   | 10       |
| A Note About Dates                             | 10       |
| A Note About Interest Rates                    | 10       |
| Statistical Analysis Functions                 | 22       |
| Date and Time Functions                        | 31       |
| Logical Functions                              | 33       |
| Informational Functions                        | 34       |
| Database Functions                             | 36       |
| Array Functions                                | 38       |
| Spreadsheet Functions                          | 39       |
| Text Functions                                 | 44       |
| Converting Decimal, Binary, Octal, Hexadecimal | 48       |
| Complex Number Functions                       | 49       |

## Functions Available in Calc

---

Calc provides all the commonly used functions found in modern spreadsheet applications. Since many of Calc's functions require very specific and carefully calculated input arguments, the descriptions in this appendix should not be considered complete references for each function. Refer to the application Help or the OpenOffice wiki for details and examples of all functions. On the wiki, start with [https://wiki.openoffice.org/wiki/Documentation/How\\_Tos/Calc:\\_Functions\\_listed\\_by\\_category](https://wiki.openoffice.org/wiki/Documentation/How_Tos/Calc:_Functions_listed_by_category)

Over 300 standard functions are available in Calc. More can be added through extensions to Calc (see Chapter 14). The following tables list Calc's functions organized into eleven categories.

### Note

Functions whose names end with **\_ADD** are provided for compatibility with Microsoft Excel functions. They return the same results as the corresponding functions in Excel (without the suffix), which, though they may be correct, are not based on international standards.

---

## Terminology: Numbers and Arguments

Some of the descriptions in this appendix define limitations on the number of values or arguments that can be passed to the function. Specifically, functions that refer to the following arguments may lead to confusion.

- Number\_1; number\_2;... number\_30
- Numbers 1 to 30
- a list of up to 30 numbers

There is a significant difference between a *list of numbers* (or integers) and the *number of arguments* a function will accept. For example, the *SUM* function will accept a maximum of 30 arguments. This limit does NOT mean that you can only sum 30 numbers, but that you can only pass 30 separate arguments to the function.

Arguments are values separated by semicolons and can include ranges that refer to multiple values. Therefore, one argument can refer to several values, and a function that limits input to 30 arguments may accept more than 30 separate numerical values. For example, *SUM(A1:A20;B11:B30)* sums 40 values using two arguments.

This appendix attempts to clarify this situation by using the term **arguments**, rather than any of the other phrases.

## Mathematical Functions

---

Table 1: Mathematical functions

| Syntax        | Description                                                                  |
|---------------|------------------------------------------------------------------------------|
| ABS(number)   | Returns the absolute value of the given <b>number</b> .                      |
| ACOS(number)  | Returns the inverse cosine of the given <b>number</b> in radians.            |
| ACOSH(number) | Returns the inverse hyperbolic cosine of the given <b>number</b> in radians. |

| <b>Syntax</b>                       | <b>Description</b>                                                                                                                                                                                                                                                                                                                       |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ACOT(number)                        | Returns the inverse cotangent of the given <b>number</b> in radians.                                                                                                                                                                                                                                                                     |
| ACOTH(number)                       | Returns the inverse hyperbolic cotangent of the given <b>number</b> in radians.                                                                                                                                                                                                                                                          |
| ASIN(number)                        | Returns the inverse sine of the given <b>number</b> in radians.                                                                                                                                                                                                                                                                          |
| ASINH(number)                       | Returns the inverse hyperbolic sine of the given <b>number</b> in radians.                                                                                                                                                                                                                                                               |
| ATAN(number)                        | Returns the inverse tangent of the given <b>number</b> in radians.                                                                                                                                                                                                                                                                       |
| ATAN2(number_x; number_y)           | Returns the inverse tangent of the specified x and y coordinates. <b>Number_x</b> is the value for the x coordinate. <b>Number_y</b> is the value for the y coordinate.                                                                                                                                                                  |
| ATANH(number)                       | Returns the inverse hyperbolic tangent of the given <b>number</b> . (Angle is returned in radians.)                                                                                                                                                                                                                                      |
| BESSELI(x; n)                       | Calculates the modified Bessel function $I_n(x)$ . <b>x</b> is the value on which the function will be calculated. <b>n</b> is the order of the Bessel function.                                                                                                                                                                         |
| BESSELJ(x; n)                       | Calculates the Bessel function $J_n(x)$ (cylinder function). <b>x</b> is the value on which the function will be calculated. <b>n</b> is the order of the Bessel function.                                                                                                                                                               |
| BESSELK(x; n)                       | Calculates the modified Bessel function $K_n(x)$ . <b>x</b> is the value on which the function will be calculated. <b>n</b> is the order of the Bessel function.                                                                                                                                                                         |
| BESSELY(x; n)                       | Calculates the modified Bessel function $Y_n(x)$ , also known as the Weber or Neumann function. <b>x</b> is the value on which the function will be calculated. <b>n</b> is the order of the Bessel function.                                                                                                                            |
| CEILING(number; significance; mode) | Rounds the given <b>number</b> to the nearest integer or multiple of significance. <b>Significance</b> is the value to whose multiple the value is to be rounded up. <b>Mode</b> is an optional value. If it is indicated and non-zero, and if the number and significance are negative, rounding up is carried out based on that value. |
| COMBIN(count_1; count_2)            | Returns the number of combinations for a given number of objects. <b>Count_1</b> is the total number of elements. <b>Count_2</b> is the selected count from the elements.                                                                                                                                                                |
| COMBINA(count_1; count_2)           | Returns the number of combinations for a given number of objects (repetition included). <b>Count_1</b> is the total number of elements. <b>Count_2</b> is the selected count from the elements.                                                                                                                                          |

| <b>Syntax</b>                                          | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CONVERT(value; "text"; "text")                         | Converts a currency value of a European currency into Euros. <b>Value</b> is the amount in the currency to be converted. <b>Text</b> is the official abbreviation for the currency in question (for example, "EUR"). The first <b>Text</b> parameter gives the source value to be converted; the second <b>Text</b> parameter gives the destination value. Both <b>text</b> arguments must be within quotes.                                             |
| CONVERT_ADD(number; from_unit; to_unit)                | Converts a value from one unit of measure to the corresponding value in another unit of measure. <b>Number</b> is the number to be converted. <b>From_unit</b> is the unit from which conversion is taking place. <b>To_unit</b> is the unit to which the conversion is taking place.                                                                                                                                                                    |
| COS(number)                                            | Returns the cosine of the given <b>number</b> (angle in radians).                                                                                                                                                                                                                                                                                                                                                                                        |
| COSH(number)                                           | Returns the hyperbolic cosine of the given <b>number</b> (angle in radians).                                                                                                                                                                                                                                                                                                                                                                             |
| COT(number)                                            | Returns the cotangent of the given <b>number</b> (angle in radians).                                                                                                                                                                                                                                                                                                                                                                                     |
| COTH(number)                                           | Returns the hyperbolic cotangent of the given <b>number</b> (angle in radians).                                                                                                                                                                                                                                                                                                                                                                          |
| COUNTBLANK(range)                                      | Returns the number of empty cells. <b>Range</b> is the cell range in which the empty cells are counted.                                                                                                                                                                                                                                                                                                                                                  |
| COUNTIF(range; criteria)                               | Returns the number of elements that meet specific criteria within a cell range. <b>Range</b> is the range to which the criteria are to be applied. <b>Criteria</b> indicates the criteria in the form of a number, a regular expression, or a character string by which the cells are counted. The character string may express a mathematical condition like ">= 5".                                                                                    |
| COUNTIFS(range 1; criteria 1; range 2; criteria 2;...) | Checks criteria along two or more cell ranges and counts the range positions where all of the criteria are met. If in the nth position of every cell range, all of the criteria are true, the count increases by one. Each <b>range X</b> is a range where <b>criteria X</b> is tested. The criteria may be in the form of a number, a regular expression, or a character string. The character string may express a mathematical condition like ">= 5". |
| DEGREES(number)                                        | Converts the given <b>number</b> in radians to degrees.                                                                                                                                                                                                                                                                                                                                                                                                  |
| DELTA(number_1; number_2)                              | Returns TRUE (1) if both <b>numbers</b> are equal, otherwise returns FALSE (0).                                                                                                                                                                                                                                                                                                                                                                          |

| <b>Syntax</b>                        | <b>Description</b>                                                                                                                                                                                                                                                                                                                            |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ERF(lower_limit;<br>upper_limit)     | Returns values of the Gaussian error integral. <b>Lower_limit</b> is the lower limit of the integral. <b>Upper_limit</b> (optional) is the upper limit of the integral. If this value is missing, the calculation takes places between 0 and the lower limit.                                                                                 |
| ERFC(lower_limit)                    | Returns complementary values of the Gaussian error integral between x and infinity. <b>Lower_limit</b> is the lower limit of the integral (x).                                                                                                                                                                                                |
| EVEN(number)                         | Rounds the given <b>number</b> up to the nearest even integer.                                                                                                                                                                                                                                                                                |
| EXP(number)                          | Returns <i>e</i> raised to the power of the given <b>number</b> .                                                                                                                                                                                                                                                                             |
| FACT(number)                         | Returns the factorial of the given <b>number</b> .                                                                                                                                                                                                                                                                                            |
| FACTDOUBLE(number)                   | Returns the factorial of the <b>number</b> with increments of 2. If the <b>number</b> is even, the following factorial is calculated: $n*(N-2)*(n-4)*...*4*2$ . If the <b>number</b> is uneven, the following factorial is calculated: $n*(N-2)*(n-4)*...*3*1$ .                                                                              |
| FLOOR(number;<br>significance; mode) | Rounds the given <b>number</b> down to the nearest multiple of <b>significance</b> . <b>Significance</b> is the value to whose multiple the number is to be rounded down. <b>Mode</b> is an optional value. If it is indicated and non-zero, and if the number and significance are negative, rounding up is carried out based on that value. |
| GCD(numbers)                         | Returns the greatest common divisor of one or more integers. <b>Numbers</b> is a list of up to 30 numbers whose greatest common divisor is to be calculated, separated by semicolons. <b>Numbers</b> can also be a cell range.                                                                                                                |
| GCD_ADD(numbers)                     | Returns the greatest common divisor of a list of numbers. <b>Numbers</b> is a list of up to 30 numbers separated by semicolons.                                                                                                                                                                                                               |
| GESTEP(number; step)                 | Returns 1 if <b>number</b> is greater than or equal to <b>step</b> .                                                                                                                                                                                                                                                                          |
| INT(number)                          | Rounds the given <b>number</b> down to the nearest integer.                                                                                                                                                                                                                                                                                   |
| ISEVEN(value)                        | Returns TRUE if the given <b>value</b> is an even integer, or FALSE if the <b>value</b> is odd. <i>If the value is not an integer, the function evaluates only the integer part of the value.</i>                                                                                                                                             |
| ISODD(value)                         | Returns TRUE if the given <b>value</b> is an odd integer, or FALSE if the <b>value</b> is even. <i>If the value is not an integer, the function evaluates only the integer part of the value.</i>                                                                                                                                             |

| <b>Syntax</b>                             | <b>Description</b>                                                                                                                                                                                                              |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LCM(integer_1; integer_2; ... integer_30) | Returns the least common multiple of one or more integers. <b>Integer_1; integer_2;... integer_30</b> are integers whose lowest common multiple is to be calculated. The <b>integer_x</b> arguments can also be a cell range.   |
| LCM_ADD(numbers)                          | <b>Numbers</b> is a list of up to 30 numbers separated by semicolons. The result is the lowest common multiple of a list of numbers.                                                                                            |
| LN(number)                                | Returns the natural logarithm based on the constant <i>e</i> of the given <b>number</b> .                                                                                                                                       |
| LOG(number; base)                         | Returns the logarithm of the given <b>number</b> to the specified base. <b>Base</b> is the base for the logarithm calculation.                                                                                                  |
| LOG10(number)                             | Returns the base-10 logarithm of the given <b>number</b> .                                                                                                                                                                      |
| MOD(dividend; divisor)                    | Returns the remainder after a number is divided by a divisor. <b>Dividend</b> is the number that will be divided by the divisor. <b>Divisor</b> is the number by which to divide the dividend.                                  |
| MROUND(number; multiple)                  | Rounds <b>number</b> to the nearest integer multiple of <b>multiple</b> .                                                                                                                                                       |
| MULTINOMIAL (number(s))                   | Returns the factorial of the sum of the arguments divided by the product of the factorials of the arguments. <b>Number(s)</b> is a list of up to 30 numbers separated by semicolons.                                            |
| ODD(number)                               | Rounds the given <b>number</b> up to the nearest odd integer.                                                                                                                                                                   |
| PI()                                      | Returns the value of PI to fourteen decimal places.                                                                                                                                                                             |
| POWER(base; power)                        | Returns the result of a number raised to a power. <b>Base</b> is the number that is to be raised to the given power. <b>Power</b> is the exponent by which the base is to be raised.                                            |
| PRODUCT(number 1 to 30)                   | Multiplies all the numbers given as arguments and returns the product. <b>Number 1 to number 30</b> are up to <i>30 arguments</i> whose product is to be calculated, separated by semicolons. The arguments may be cell ranges. |
| QUOTIENT(numerator; denominator)          | Returns the integer result of a division operation. <b>Numerator</b> is the number that will be divided. <b>Denominator</b> is the number the numerator will be divided by.                                                     |
| RADIANS(number)                           | Converts the given <b>number</b> in degrees to radians.                                                                                                                                                                         |
| RAND()                                    | Returns a random number between 0 and 1. This number will recalculate every time data is entered or F9 is pressed.                                                                                                              |



| <b>Syntax</b>                    | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RANDBETWEEN (bottom; top)        | Returns an integer random number between <b>bottom</b> and <b>top</b> (inclusive). This number will recalculate when the <i>Control+Shift+F9</i> key combination is pressed.                                                                                                                                                                                                                                                                                                                                                                                                              |
| ROUND(number; count)             | Rounds the given <b>number</b> to a certain number of decimal places according to valid mathematical criteria. <b>Count</b> (optional) is the number of decimal places to which the value is to be rounded. If the count parameter is negative, the number is rounded to a multiple of 10, 100, 1000, etc..                                                                                                                                                                                                                                                                               |
| ROUNDDOWN(number; count)         | Rounds the given <b>number</b> . <b>Count</b> (optional) is the number of decimal places to be rounded down to. If the <b>count</b> parameter is negative, the number is rounded to a multiple of 10, 100, 1000, etc..                                                                                                                                                                                                                                                                                                                                                                    |
| ROUNDUP(number; count)           | Rounds the given <b>number</b> up. <b>Count</b> (optional) is the number of decimal places to which rounding up is to be done. If the count parameter is negative, the number is rounded to a multiple of 10, 100, 1000, etc..                                                                                                                                                                                                                                                                                                                                                            |
| SERIESSUM(x; n; m; coefficients) | <p>Returns a sum of powers of the number x in accordance with the following formula:</p> $\text{SERIESSUM}(x;n;m;\text{coefficients}) = \text{coefficient}_1 * x^n + \text{coefficient}_2 * x^{(n+m)} + \text{coefficient}_3 * x^{(n+2m)} + \dots + \text{coefficient}_i * x^{(n+(i-1)m)}.$ <p><b>x</b> is the number as an independent variable. <b>n</b> is the starting power. <b>m</b> is the increment. <b>Coefficients</b> is a series of coefficients. For each coefficient, the series sum is extended by one section. You can only enter coefficients using cell references.</p> |
| SIGN(number)                     | Returns the sign of the given <b>number</b> . The function returns the result 1 for a positive sign, -1 for a negative sign, and 0 for zero.                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| SIN(number)                      | Returns the sine of the given <b>number</b> (angle in radians).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| SINH(number)                     | Returns the hyperbolic sine of the given <b>number</b> (angle in radians).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| SQRT(number)                     | Returns the positive square root of the given <b>number</b> . The value of the <b>number</b> must be positive.                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| SQRTPI(number)                   | Returns the square root of the product of the given <b>number</b> and PI.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| SUBTOTAL(function; range)        | Calculates subtotals defined by cells that are visible after using a filter. If a range already contains subtotals, these are not used for further calculations. <b>Function</b> is a value that stands for another function, such as Average, Count, Min, Sum, and Var. <b>Range</b> is the range whose cells are included.                                                                                                                                                                                                                                                              |

| <b>Syntax</b>                                                    | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SUM(number_1; number_2; ... number_30)                           | Adds all the numbers in a range of cells. <b>Number_1; number_2;... number_30</b> are up to 30 arguments whose sum is to be calculated. You can also enter a range using cell references.                                                                                                                                                                                                                |
| SUMIF(range; criteria; sum_range)                                | Adds the cells specified by a given criteria. The search supports regular expressions. <b>Range</b> is the range to which the criteria are to be applied. <b>Criteria</b> is the cell in which the search criterion is shown, or the search criterion itself. <b>Sum_range</b> is the range from which values are summed; if it has not been indicated, the values found in the <b>Range</b> are summed. |
| SUMIFS(sum_range; range 1; criteria 1; range 2; criteria 2; ...) | Returns the sum of <b>sum_range</b> using only those positions where every <b>range X</b> meets <b>criteria X</b> .                                                                                                                                                                                                                                                                                      |
| SUMSQ(number_1; number_2; ... number_30)                         | Calculates the sum of the squares of numbers (totaling up of the squares of the arguments) <b>Number_1; number_2;... number_30</b> are up to 30 arguments, the sum of whose squares is to be calculated.                                                                                                                                                                                                 |
| TAN(number)                                                      | Returns the tangent of the given <b>number</b> (angle in radians).                                                                                                                                                                                                                                                                                                                                       |
| TANH(number)                                                     | Returns the hyperbolic tangent of the given <b>number</b> (angle in radians).                                                                                                                                                                                                                                                                                                                            |
| TRUNC(number; count)                                             | Truncates a number by removing the digits after the <b>count</b> decimal places. Cell formatting or settings specified in <b>Tools &gt; Options &gt; OpenOffice Calc &gt; Calculate</b> may limit the <i>display</i> of the number to fewer decimal places. <b>Number</b> is the number whose decimal places are to be cut off. <b>Count</b> is the number of decimal places that are not cut off.       |

## Financial Analysis Functions

### A Note About Dates

Date values are used in several financial functions. They can be entered in the format "YYYY-MM-DD", e.g., "2021-03-31". For more flexibility with date formats, you can enter the date in a cell and use the cell reference in the financial function.

### A Note About Interest Rates

You can enter interest rates in either of two ways:

- As a decimal: To enter an interest rate as a decimal, divide it by 100 before entering it into a function. For example, to compute a loan with a 3.25% interest rate, enter .0325 into the function.

- As a percentage: To enter an interest rate as a percentage, type in the interest rate followed by the % key. For example, to compute a loan with a 3.25% interest rate, enter 3.25% into the function.

If you enter it as 3.25, the function will treat it as a 325% interest rate.

Accounting systems vary in the number of days in a month or a year used for calculations. The following table lists the integers used for the **basis** parameter in some financial analysis functions.

Table 2: Basis calculation types

| <b>Basis</b> | <b>Calculation</b>                                                |
|--------------|-------------------------------------------------------------------|
| 0 or missing | US method (NASD), 12 months of 30 days each, a year has 360 days. |
| 1            | Exact number of days in each month and year.                      |
| 2            | Exact number of days in a month, a year has 360 days.             |
| 3            | Exact number of days in a month, a year has 365 days.             |
| 4            | European method, 12 months of 30 days each.                       |

Table 3: Financial analysis functions

| <b>Syntax</b>                                                               | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ACCRINT(issue; first_interest; settlement; rate; par; frequency; basis)     | Calculates the accrued interest of a security in the case of periodic payments. <b>Issue</b> is the issue date of the security. <b>First_interest</b> is the first interest date of the security. <b>Settlement</b> is the maturity date. <b>Rate</b> is the annual nominal rate of interest (coupon interest rate). <b>Par</b> is the par value of the security. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                              |
| ACCRINTM(issue; settlement; rate; par; basis)                               | Calculates the accrued interest of a security in the case of a one-off payment at the settlement date. <b>Issue</b> is the issue date of the security. <b>Settlement</b> is the maturity date. <b>Rate</b> is the annual nominal rate of interest (coupon interest rate). <b>Par</b> is the par value of the security. <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                                                                    |
| AMORDEGRC(cost; date_purchased; first_period; salvage; period; rate; basis) | Calculates the amount of depreciation for a settlement period as degressive amortization. Unlike AMORLINC, a depreciation coefficient that is independent of the depreciable life is used here. <b>Cost</b> is the acquisition cost. <b>Date_purchased</b> is the date of acquisition. <b>First_period</b> is the end date of the first settlement period. <b>Salvage</b> is the salvage value of the capital asset at the end of the depreciable life. <b>Period</b> is the settlement period to be considered. <b>Rate</b> is the rate of depreciation. <b>Basis</b> indicates how the year is to be calculated. |

| <b>Syntax</b>                                                                    | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AMORLINC(cost;<br>date_purchased; first_period;<br>salvage; period; rate; basis) | Calculates the amount of depreciation for a settlement period as linear amortization. If the capital asset is purchased during the settlement period, the proportional amount of depreciation is considered. <b>Cost</b> is the acquisition cost. <b>Date_purchased</b> is the date of acquisition. <b>First_period</b> is the end date of the first settlement period. <b>Salvage</b> is the salvage value of the capital asset at the end of the depreciable life. <b>Period</b> is the settlement period to be considered. <b>Rate</b> is the rate of depreciation. <b>Basis</b> indicates how the year is to be calculated. |
| COUPDAYBS(settlement;<br>maturity; frequency; basis)                             | Returns the number of days from the first day of interest payment on a security until the settlement date. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                                                                                  |
| COUPDAYS(settlement;<br>maturity; frequency; basis)                              | Returns the number of days in the current interest period in which the settlement date falls. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                                                                                               |
| COUPDAYSNC(settlement;<br>maturity; frequency; basis)                            | Returns the number of days from the settlement date until the next interest date. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                                                                                                           |
| COUPNCD(settlement;<br>maturity; frequency; basis)                               | Returns the date of the first interest date after the settlement date, and formats the result as a date. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                                                                                    |
| COUPNUM(settlement;<br>maturity; frequency; basis)                               | Returns the number of coupons (interest payments) between the settlement date and the maturity date. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                                                                                        |

| <b>Syntax</b>                                                | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| COUPPCD(settlement; maturity; frequency; basis)              | Returns the date of the interest date prior to the settlement date, and formats the result as a date. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                                                                              |
| CUMIPMT(rate; NPER; PV; S; E; type)                          | Calculates the cumulative interest payments (the total interest) for an investment based on a constant interest rate. <b>Rate</b> is the periodic interest rate. <b>NPER</b> is the payment period with the total number of periods. <b>NPER</b> can also be a non-integer value. The <b>rate</b> and <b>NPER</b> must refer to the same unit, and thus both must be calculated annually or monthly. <b>PV</b> is the current value in the sequence of payments. <b>S</b> is the first period. <b>E</b> is the last period. <b>Type</b> is the due date of the payment at the beginning (1) or end (0) of each period. |
| CUMIPMT_ADD(rate; NPER; PV; start_period; end_period; type)  | Calculates the accumulated interest for a period. <b>Rate</b> is the interest rate for each period. <b>NPER</b> is the total number of payment periods. The <b>rate</b> and <b>NPER</b> must refer to the same unit, and thus both must be calculated annually or monthly. <b>PV</b> is the current value. <b>Start_period</b> is the first payment period for the calculation. <b>End_period</b> is the last payment period for the calculation. <b>Type</b> is the due date of the payment at the beginning (1) or end (0) of each period.                                                                           |
| CUMPRINC(rate; NPER; PV; S; E; type)                         | Returns the cumulative principal paid for an investment period with a constant interest rate. <b>Rate</b> is the periodic interest rate. <b>NPER</b> is the payment period with the total number of periods. <b>NPER</b> can also be a non-integer value. The <b>rate</b> and <b>NPER</b> must refer to the same unit, and thus both must be calculated annually or monthly. <b>PV</b> is the current value in the sequence of payments. <b>S</b> is the first period. <b>E</b> is the last period. <b>Type</b> is the due date of the payment at the beginning (1) or end (0) of each period.                         |
| CUMPRINC_ADD(rate; NPER; PV; start_period; end_period; type) | Calculates the cumulative redemption of a loan in a period. <b>Rate</b> is the interest rate for each period. <b>NPER</b> is the total number of payment periods. The <b>rate</b> and <b>NPER</b> must refer to the same unit, and thus both must be calculated annually or monthly. <b>PV</b> is the current value. <b>Start period</b> is the first payment period for the calculation. <b>End period</b> is the last payment period for the calculation. <b>Type</b> is the due date of the payment at the beginning (1) or end (0) of each period.                                                                 |

| <b>Syntax</b>                                        | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DB(cost; salvage; life; period; month)               | Returns the depreciation of an asset for a specified period using the double-declining balance method. <b>Cost</b> is the initial cost of an asset. <b>Salvage</b> is the value of an asset at the end of the depreciation. <b>Life</b> defines the period over which an asset is depreciated. <b>Period</b> is the length of each period. The life must be entered in the same date unit as the depreciation period. <b>Month</b> (optional) denotes the number of months for the first year of depreciation.                                                          |
| DDB(cost; salvage; life; period; factor)             | Returns the depreciation of an asset for a specified period using the arithmetic-declining method. Note that the book value will never reach zero under this calculation type. <b>Cost</b> fixes the initial cost of an asset. <b>Salvage</b> fixes the value of an asset at the end of its life. <b>Life</b> is the number of periods defining how long the asset is to be used. <b>Period</b> defines the length of the period. The period must be entered in the same time unit as the life. <b>Factor</b> (optional) is the factor by which depreciation decreases. |
| DISC(settlement; maturity; price; redemption; basis) | Calculates the allowance (discount) of a security as a percentage. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Price</b> is the price of the security per 100 currency units of par value. <b>Redemption</b> is the redemption value of the security per 100 currency units of par value. <b>Basis</b> indicates how the year is to be calculated.                                                                                                                               |
| DOLLARDE(fractional _dollar; fraction)               | Converts a quotation that has been given as a decimal fraction into a decimal number. <b>Fractional_dollar</b> is a number given as a decimal fraction. (In this number, the decimal value is the numerator of the fraction.) <b>Fraction</b> is a whole number that is used as the denominator of the decimal fraction.                                                                                                                                                                                                                                                |
| DOLLARFR(decimal _dollar; fraction)                  | Converts a quotation that has been given as a decimal number into a mixed decimal fraction. The decimal of the result is the numerator of the fraction that would have <b>Fraction</b> as the denominator. <b>Decimal_dollar</b> is a decimal number. <b>Fraction</b> is a whole number that is used as the denominator of the decimal fraction.                                                                                                                                                                                                                        |
| DURATION(rate; PV; FV)                               | Calculates the number of periods required by an investment to attain the desired value. <b>Rate</b> (a constant) is the interest rate to be used for the entire duration. <b>PV</b> is the present value. <b>FV</b> is the desired future value of the investment.                                                                                                                                                                                                                                                                                                      |

| <b>Syntax</b>                                                        | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DURATION_ADD (settlement; maturity; coupon; yield; frequency; basis) | Calculates the duration of a fixed interest security in years. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Coupon</b> is the annual coupon interest rate (nominal rate of interest). <b>Yield</b> is the annual yield of the security. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated. |
| EFFECT_ADD(nominal_rate; Npery)                                      | Calculates the effective annual rate of interest on the basis of the nominal interest rate and the number of interest payments per annum. Nominal interest refers to the amount of interest due at the end of a calculation period. <b>Nominal_rate</b> is the annual nominal rate of interest. <b>Npery</b> is the number of interest payments per year.                                                                                                         |
| EFFECTIVE(NOM; P)                                                    | Calculates the effective annual rate of interest on the basis of the nominal interest rate and the number of interest payments per annum. Nominal interest refers to the amount of interest due at the end of a calculation period. <b>NOM</b> is the nominal interest. <b>P</b> is the number of interest payment periods per year.                                                                                                                              |
| FV(rate; NPER; PMT; PV; type)                                        | Returns the future value of an investment based on periodic, constant payments and a constant interest rate. <b>Rate</b> is the periodic interest rate. <b>NPER</b> is the total number of periods. <b>PMT</b> is the annuity paid regularly per period. <b>PV</b> (optional) is the present cash value of an investment. <b>Type</b> (optional) defines whether the payment is due at the beginning (1) or the end (0) of a period.                              |
| FVSCHEDULE(principal; schedule)                                      | Calculates the accumulated value of the starting capital for a series of periodically varying interest rates. <b>Principal</b> is the starting capital. <b>Schedule</b> is a series of interest rates. <b>Schedule</b> has to be entered with cell references.                                                                                                                                                                                                    |
| INTRATE(settlement; maturity; investment; redemption; basis)         | Calculates the annual interest rate that results when a security (or other item) is purchased at an investment value and sold at a redemption value with no interest being paid. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security is sold. <b>Investment</b> is the purchase price. <b>Redemption</b> is the selling price. <b>Basis</b> indicates how the year is to be calculated.                  |

| <b>Syntax</b>                                                    | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| IPMT(rate; period; NPER; PV; FV; type)                           | Calculates the periodic amortization for an investment with regular payments and a constant interest rate. <b>Rate</b> is the periodic interest rate. <b>Period</b> is the period for which the compound interest is calculated. <b>NPER</b> is the total number of periods during which the annuity is paid. <b>Period=NPER</b> , if compound interest for the last period is calculated. <b>PV</b> is the present cash value in the sequence of payments. <b>FV</b> (optional) is the desired value (future value) at the end of the periods. <b>Type</b> (optional) defines whether the payment is due at the beginning (1) or the end (0) of a period. |
| IRR(values; guess)                                               | Calculates the internal rate of return for an investment. The values represent cash flow values at regular intervals; at least one value must be negative (payments), and at least one value must be positive (income). <b>Values</b> is an array containing the values. <b>Guess</b> (optional) is the estimated value. If you can provide only a few values, you should provide an initial guess to enable the iteration.                                                                                                                                                                                                                                |
| ISPMT(rate; period; total_periods; invest)                       | Calculates the level of interest for unchanged amortization installments. <b>Rate</b> sets the periodic interest rate. <b>Period</b> is the number of installments for calculation of interest. <b>Total_periods</b> is the total number of installment periods. <b>Invest</b> is the amount of the investment.                                                                                                                                                                                                                                                                                                                                            |
| MDURATION(settlement; maturity; coupon; yield; frequency; basis) | Calculates the modified Macauley duration of a fixed interest security in years. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Coupon</b> is the annual nominal rate of interest (coupon interest rate) <b>Yield</b> is the annual yield of the security. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                         |
| MIRR(values; investment; reinvest_rate)                          | Calculates the modified internal rate of return of a series of investments. <b>Values</b> corresponds to the array or the cell reference for cells whose content corresponds to the payments. <b>Investment</b> is the rate of interest of the investments (the negative values of the array) <b>Reinvest_rate</b> is the rate of interest of the reinvestment (the positive values of the array).                                                                                                                                                                                                                                                         |



| <b>Syntax</b>                                                                                   | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NOMINAL(effective_rate; Npery)                                                                  | Calculates the yearly nominal interest rate, given the effective rate and the number of compounding periods per year. <b>Effective_rate</b> is the effective interest rate <b>Npery</b> is the number of periodic interest payments per year.                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| NOMINAL_ADD(effective_rate ; Npery)                                                             | Calculates the yearly nominal rate of interest, given the effective rate and the number of compounding periods per year. <b>Effective_rate</b> is the effective annual rate of interest. <b>Npery</b> is the number of interest payments per year.                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| NPER(rate; PMT; PV; FV; type)                                                                   | Returns the number of periods for an investment based on periodic, constant payments and a constant interest rate. <b>Rate</b> is the periodic interest rate. <b>PMT</b> is the constant annuity paid in each period. <b>PV</b> is the present value (cash value) in a sequence of payments. <b>FV</b> (optional) is the future value, which is reached at the end of the last period. <b>Type</b> (optional) defines whether the payment is due at the beginning (1) or the end (0) of a period.                                                                                                                                                                                      |
| NPV(Rate; value_1; value_2; ... value_30)                                                       | Returns the net present value of an investment based on a series of periodic cash flows and a discount rate. <b>Rate</b> is the discount rate for a period. <b>Value_1; value_2;... value_30</b> are values representing deposits or withdrawals.                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| ODDFPRICE(settlement; maturity; issue; first_coupon; rate; yield; redemption; frequency; basis) | Calculates the price per 100 currency units par value of a security, if the first interest date falls irregularly. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Issue</b> is the date of issue of the security. <b>First_coupon</b> is the first interest date of the security. <b>Rate</b> is the annual rate of interest. <b>Yield</b> is the annual yield of the security. <b>Redemption</b> is the redemption value per 100 currency units of par value. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated. |
| ODDFYIELD(settlement; maturity; issue; first_coupon; rate; price; redemption; frequency; basis) | Calculates the yield of a security if the first interest date falls irregularly. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Issue</b> is the date of issue of the security. <b>First_coupon</b> is the first interest period of the security. <b>Rate</b> is the annual rate of interest. <b>Price</b> is the price of the security. <b>Redemption</b> is the redemption value per 100 currency units of par value. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                        |

| <b>Syntax</b>                                                                             | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ODDLPRICE(settlement; maturity; last_interest; rate; yield; redemption; frequency; basis) | Calculates the price per 100 currency units par value of a security, if the last interest date falls irregularly. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Last_interest</b> is the last interest date of the security. <b>Rate</b> is the annual rate of interest. <b>Yield</b> is the annual yield of the security. <b>Redemption</b> is the redemption value per 100 currency units of par value. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated. |
| ODDLYIELD(settlement; maturity; last_interest; rate; price; redemption; frequency; basis) | Calculates the yield of a security if the last interest date falls irregularly. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Last_interest</b> is the last interest date of the security. <b>Rate</b> is the annual rate of interest. <b>Price</b> is the price of the security. <b>Redemption</b> is the redemption value per 100 currency units of par value. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                          |
| PMT(rate; NPER; PV; FV; type)                                                             | Returns the periodic payment for an annuity with constant interest rates. <b>Rate</b> is the periodic interest rate. <b>NPER</b> is the number of periods in which annuity is paid. <b>PV</b> is the present value (cash value) in a sequence of payments. <b>FV</b> (optional) is the desired value (future value) to be reached at the end of the periodic payments. <b>Type</b> (optional) defines whether the payment is due at the beginning (1) or the end (0) of a period.                                                                                                                                                  |
| PPMT(rate; period; NPER; PV; FV; type)                                                    | Returns for a given period the payment on the principal for an investment that is based on periodic and constant payments and a constant interest rate. <b>Rate</b> is the periodic interest rate. <b>Period</b> is the amortization period. <b>NPER</b> is the total number of periods during which annuity is paid. <b>PV</b> is the present value in the sequence of payments. <b>FV</b> (optional) is the desired (future) value. <b>Type</b> (optional) defines whether the payment is due at the beginning (1) or the end (0) of a period.                                                                                   |

| <b>Syntax</b>                                                          | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PRICE(settlement; maturity; rate; yield; redemption; frequency; basis) | Calculates the market value of a fixed interest security with a par value of 100 currency units as a function of the forecast yield. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Rate</b> is the annual nominal rate of interest (coupon interest rate). <b>Yield</b> is the annual yield of the security. <b>Redemption</b> is the redemption value per 100 currency units of par value. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated. |
| PRICEDISC(settlement; maturity; discount; redemption; basis)           | Calculates the price per 100 currency units of par value of a non-interest-bearing security. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Discount</b> is the discount of a security as a percentage. <b>Redemption</b> is the redemption value per 100 currency units of par value. <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                  |
| PRICEMAT(settlement; maturity; issue; rate; yield; basis)              | Calculates the price per 100 currency units of par value of a security that pays interest on the maturity date. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Issue</b> is the date of issue of the security. <b>Rate</b> is the interest rate of the security on the issue date. <b>Yield</b> is the annual yield of the security. <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                    |
| PV(rate; NPER; PMT; FV; type)                                          | Returns the present value of an investment resulting from a series of regular payments. <b>Rate</b> defines the interest rate per period. <b>NPER</b> is the total number of payment periods. <b>PMT</b> is the regular payment made per period. <b>FV</b> (optional) defines the future value remaining after the final installment has been made. <b>Type</b> (optional) defines whether the payment is due at the beginning (1) or the end (0) of a period.                                                                                                                                                       |
| RATE(NPER; PMT; PV; FV; type; guess)                                   | Returns the constant interest rate per period of an annuity. <b>NPER</b> is the total number of periods during which payments are made (payment period). <b>PMT</b> is the constant payment (annuity) paid during each period. <b>PV</b> is the cash value in the sequence of payments. <b>FV</b> (optional) is the future value, which is reached at the end of the periodic payments. <b>Type</b> (optional) defines whether the payment is due at the beginning (1) or the end (0) of a period. <b>Guess</b> (optional) determines the estimated value of the interest with iterative calculation.                |

| <b>Syntax</b>                                               | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RECEIVED(settlement; maturity; investment; discount; basis) | Calculates the amount received that is paid for a fixed-interest security at a given point in time. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures. <b>Investment</b> is the purchase sum. <b>Discount</b> is the percentage discount on acquisition of the security. <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                        |
| RRI(P; PV; FV)                                              | Calculates the interest rate resulting from the profit (return) of an investment. <b>P</b> is the number of periods needed for calculating the interest rate. <b>PV</b> is the present value (must be >0). <b>FV</b> determines what is desired as the cash value of the deposit.                                                                                                                                                                                                                                                                           |
| SLN(cost; salvage; life)                                    | Returns the straight-line depreciation of an asset for one period. The amount of the depreciation is constant during the depreciation period. <b>Cost</b> is the initial cost of an asset. <b>Salvage</b> is the value of an asset at the end of the depreciation. <b>Life</b> is the depreciation period determining the number of periods in the depreciation of the asset.                                                                                                                                                                               |
| SYD(cost; salvage; life; period)                            | Returns the arithmetic-declining depreciation rate. Use this function to calculate the depreciation amount for one period of the total depreciation span of an object. Arithmetic declining depreciation reduces the depreciation amount from period to period by a fixed sum. <b>Cost</b> is the initial cost of an asset. <b>Salvage</b> is the value of an asset after depreciation. <b>Life</b> is the period fixing the time span over which an asset is depreciated. <b>Period</b> defines the period for which the depreciation is to be calculated. |
| TBILLEQ(settlement; maturity; discount)                     | Calculates the annual return on a treasury bill. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Discount</b> is the percentage discount on acquisition of the security.                                                                                                                                                                                                                                                                                                 |
| TBILLPRICE(settlement; maturity; discount)                  | Calculates the price of a treasury bill per 100 currency units. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Discount</b> is the percentage discount upon acquisition of the security.                                                                                                                                                                                                                                                                                |
| TBILLYIELD(settlement; maturity; price)                     | Calculates the yield of a treasury bill. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Price</b> is the price (purchase price) of the treasury bill per 100 currency units of par value.                                                                                                                                                                                                                                                                               |

| <b>Syntax</b>                                                          | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VDB(cost; salvage; life; start; end; factor; type)                     | Returns the depreciation of an asset for a specified or partial period using a variable declining balance method. <b>Cost</b> is the initial value of an asset. <b>Salvage</b> is the value of an asset at the end of the depreciation. <b>Life</b> is the depreciation duration of the asset. <b>Start</b> is the start of the depreciation entered in the same date unit as the life. <b>End</b> is the end of the depreciation. <b>Factor</b> (optional) is the depreciation factor. FA=2 is double rate depreciation. <b>Type</b> (optional) defines whether the payment is due at the beginning (1) or the end (0) of a period. |
| XIRR(values; dates; guess)                                             | Calculates the internal rate of return for a list of payments that take place on different dates. The calculation is based on a 365-day per year basis, ignoring leap years. If the payments take place at regular intervals, use the IRR function. <b>Values</b> and <b>dates</b> are a series of payments and the series of associated date values entered as cell references. <b>Guess</b> (optional) is a guess for the internal rate of return. The default is 10%.                                                                                                                                                             |
| XNPV(rate; values; dates)                                              | Calculates the capital value (net present value) for a list of payments that take place on different dates. The calculation is based on a 365-day per year basis, ignoring leap years. If the payments take place at regular intervals, use the NPV function. <b>Rate</b> is the internal rate of return for the payments. <b>Values</b> and <b>dates</b> are a series of payments and the series of associated date values entered as cell references.                                                                                                                                                                              |
| YIELD(settlement; maturity; rate; price; redemption; frequency; basis) | Calculates the yield of a security. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Rate</b> is the annual rate of interest. <b>Price</b> is the price (purchase price) of the security per 100 currency units of par value. <b>Redemption</b> is the redemption value per 100 currency units of par value. <b>Frequency</b> is the number of interest payments per year (1, 2, or 4). <b>Basis</b> indicates how the year is to be calculated.                                                                                                   |
| YIELDDISC(settlement; maturity; price; redemption; basis)              | Calculates the annual yield of a non-interest-bearing security. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Price</b> is the price (purchase price) of the security per 100 currency units of par value. <b>Redemption</b> is the redemption value per 100 currency units of par value. <b>Basis</b> indicates how the year is to be calculated.                                                                                                                                                                                              |

| <b>Syntax</b>                                             | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| YIELDMAT(settlement; maturity; issue; rate; price; basis) | Calculates the annual yield of a security, the interest of which is paid on the date of maturity. <b>Settlement</b> is the date of purchase of the security. <b>Maturity</b> is the date on which the security matures (expires). <b>Issue</b> is the date of issue of the security. <b>Rate</b> is the interest rate of the security on the issue date. <b>Price</b> is the price (purchase price) of the security per 100 currency units of par value. <b>Basis</b> indicates how the year is to be calculated. |

## Statistical Analysis Functions

Calc includes over 70 statistical functions that enable the evaluation of data from simple arithmetic calculations, such as averaging, to advanced distribution and probability computations.

Table 4: Statistical analysis functions

| <b>Syntax</b>                              | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                     |
|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AVEDEV(number1; number2; ... number_30)    | Returns the average of the absolute deviations of data points from their mean. Displays the diffusion in a data set. <b>Number_1; number_2; ... number_30</b> are values or ranges that represent a sample. Each number can also be replaced by a reference.                                                                                                           |
| AVERAGE(number_1; number_2; ... number_30) | Returns the average of the arguments. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges. Text is ignored.                                                                                                                                                                                                                                        |
| AVERAGEA(value_1; value_2; ... value_30)   | Returns the average of the arguments. The value of a text is 0. <b>Value_1; value_2; ... value_30</b> are values or ranges.                                                                                                                                                                                                                                            |
| B(trials; SP; T_1; T_2)                    | Returns the probability of a sample with a binomial distribution. <b>Trials</b> is the number of independent trials. <b>SP</b> is the probability of success on each trial. <b>T_1</b> defines the number of successes or the lower limit for the number of successes. <b>T_2</b> (optional) defines the upper limit for the number of successes, if there is a range. |

| <b>Syntax</b>                                         | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BETADIST(number; alpha; beta; start; end; cumulative) | Returns the cumulative beta probability density function. <b>Number</b> is the value between <b>Start</b> and <b>End</b> at which to evaluate the function. <b>Alpha</b> and <b>Beta</b> are the shape parameters of the distribution. <b>Start</b> (optional) and <b>End</b> (optional) are the bounds for <b>number</b> . They expand or contract the range over which the Beta distribution is supported. The default values are 0 and 1. <b>Cumulative</b> (optional) can be 0 or False to calculate the probability density function. It can be any other value, or True, or omitted to calculate the cumulative distribution function. |
| BETAINV(number; alpha; beta; start; end)              | Returns the inverse of the cumulative beta probability density function. <b>Number</b> is the value between <b>Start</b> and <b>End</b> at which to evaluate the function. <b>Alpha</b> and <b>Beta</b> are the shape parameters of the distribution. <b>Start</b> (optional) and <b>End</b> (optional) are the bounds for number. They expand or contract the range over which the Beta distribution is supported. The default values are 0 and 1.                                                                                                                                                                                          |
| BINOMDIST(X; trials; SP; C)                           | Returns the individual term binomial distribution probability. <b>X</b> is the number of successes in a set of trials. <b>Trials</b> is the number of independent trials. <b>SP</b> is the probability of success on each trial. <b>C</b> = 0 calculates the probability of a single event, and <b>C</b> = 1 calculates the cumulative probability.                                                                                                                                                                                                                                                                                          |
| CHIDIST(number; degrees_freedom)                      | Returns the right-tailed probability of the chi-square distribution. The probability determined by CHIDIST can also be determined by CHITEST. <b>Number</b> is the chi-square value of the random sample used to determine the error probability. <b>Degrees_freedom</b> is the degrees of freedom of the experiment.                                                                                                                                                                                                                                                                                                                        |
| CHIINV(number; degrees_freedom)                       | Returns the inverse of the one-tailed probability of the chi-squared distribution. <b>Number</b> is the value of the error probability. <b>Degrees_freedom</b> is the degrees of freedom of the experiment.                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| CHITEST(data_B; data_E)                               | Returns the chi-square distribution from a random distribution of two test series based on the chi-square test for independence. The probability determined by CHITEST can also be determined with CHIDIST, in which case the chi-square of the random sample must then be passed as a parameter instead of the data rows. <b>Data_B</b> is the array of the observations. <b>Data_E</b> is the range of the expected values.                                                                                                                                                                                                                |

| <b>Syntax</b>                                       | <b>Description</b>                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CONFIDENCE(alpha; STDEV; size)                      | Returns the (1 - alpha) confidence interval for a normal distribution. <b>Alpha</b> is the level of the confidence interval. <b>STDEV</b> is the standard deviation for the total population. <b>Size</b> is the size of the total population.                                                                                              |
| CORREL(data_1; data_2)                              | Returns the correlation coefficient between two data sets. <b>Data_1</b> is the first data set. <b>Data_2</b> is the second data set.                                                                                                                                                                                                       |
| COUNT(value_1; value_2; ... value_30)               | Counts how many numbers are in the list of arguments. Text entries are ignored. <b>Value_1; value_2; ... value_30</b> are values or ranges which are to be counted.                                                                                                                                                                         |
| COUNTA(value_1; value_2; ... value_30)              | Counts how many values are in the list of arguments. Text entries are also counted, even when they contain an empty string of length 0. If an argument is an array or reference, empty cells within the array or reference are ignored. <b>value_1; value_2; ... value_30</b> are up to 30 arguments representing the values to be counted. |
| COVAR(data_1; data_2)                               | Returns the covariance of the product of paired deviations. <b>Data_1</b> is the first data set. <b>Data_2</b> is the second data set.                                                                                                                                                                                                      |
| CRITBINOM(trials; SP; alpha)                        | Returns the smallest value for which the cumulative binomial distribution is less than or equal to a criterion value. <b>Trials</b> is the total number of trials. <b>SP</b> is the probability of success for one trial. <b>Alpha</b> is the threshold probability to be reached or exceeded.                                              |
| DEVSQ(number_1; number_2; ... number_30)            | Returns the sum of squares of deviations based on a sample mean. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges representing a sample.                                                                                                                                                                             |
| EXPONDIST(number; lambda; C)                        | Returns the exponential distribution. <b>Number</b> is the value of the function. <b>Lambda</b> is the parameter value of the distribution. <b>C</b> is a logical value that determines the form of the function. <b>C</b> = 0 calculates the probability density function, and <b>C</b> = 1 calculates the distribution.                   |
| FDIST(number; degrees_freedom_1; degrees_freedom_2) | Calculates the values of an F probability distribution. <b>Number</b> is the value for which the F distribution is to be calculated.<br><b>Degrees_freedom_1</b> is the degrees of freedom in the numerator in the F distribution.<br><b>Degrees_freedom_2</b> is the degrees of freedom in the denominator in the F distribution.          |



| <b>Syntax</b>                                            | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FINV(number;<br>degrees_freedom_1;<br>degrees_freedom_2) | Returns the inverse of the F probability distribution. <b>Number</b> is probability value for which the inverse F distribution is to be calculated. <b>Degrees_freedom_1</b> is the number of degrees of freedom in the numerator of the F distribution. <b>Degrees_freedom_2</b> is the number of degrees of freedom in the denominator of the F distribution.                          |
| FISHER(number)                                           | Returns the Fisher transformation for the given <b>number</b> and creates a function close to a normal distribution.                                                                                                                                                                                                                                                                     |
| FISHERINV(number)                                        | Returns the inverse of the Fisher transformation for the given <b>number</b> and creates a function close to a normal distribution.                                                                                                                                                                                                                                                      |
| FORECAST(value; data_Y;<br>data_X)                       | Extrapolates future values based on a linear fit of existing x and y values. <b>Value</b> is the x value, for which the y value of the linear regression is to be returned. <b>Data_Y</b> is the array or range of known y's. <b>Data_X</b> is the array or range of known x's.                                                                                                          |
| FTEST(data_1; data_2)                                    | Returns the result of an F test. <b>Data_1</b> is the first record array. <b>Data_2</b> is the second record array.                                                                                                                                                                                                                                                                      |
| GAMMADIST(number; alpha;<br>beta; C)                     | Returns the values of a Gamma distribution. <b>Number</b> is the value for which the Gamma distribution is to be calculated. <b>Alpha</b> is the parameter Alpha of the Gamma distribution. <b>Beta</b> is the parameter Beta (scale) of the Gamma distribution. <b>C</b> = 0 calculates the density function, and <b>C</b> = 1, which is the default, calculates the distribution.      |
| GAMMAINV(number; alpha;<br>beta)                         | Returns the inverse of the Gamma cumulative distribution. This function allows you to search for variables with different distributions.<br><br><b>Number</b> is the probability value for which the inverse Gamma distribution is to be calculated. <b>Alpha</b> is the parameter Alpha of the Gamma distribution. <b>Beta</b> is the parameter Beta (scale) of the Gamma distribution. |
| GAMMALN(number)                                          | Returns the natural logarithm of the Gamma function, $\Gamma(x)$ , for the given <b>number</b> .                                                                                                                                                                                                                                                                                         |
| GAUSS(number)                                            | Returns the standard normal cumulative distribution, shifted by 0.5 on the y axis so it passes through (0,0), for the given <b>number</b> .                                                                                                                                                                                                                                              |
| GEOMEAN(number_1;<br>number_2; ... number_30)            | Returns the geometric mean of a sample. <b>Number_1; number_2; ... number_30</b> are numerical arguments or ranges that represent a random sample.                                                                                                                                                                                                                                       |

| <b>Syntax</b>                                     | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                          |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HARMEAN(number_1; number_2; ... number_30)        | Returns the harmonic mean of a data set. <b>Number_1; number_2; ... number_30</b> are values or ranges that can be used to calculate the harmonic mean.                                                                                                                                                                                                                                     |
| HYPGEOMDIST(X; n_sample; successes; n_population) | Returns the hypergeometric distribution. <b>X</b> is the number of results achieved in the random sample. <b>N_sample</b> is the size of the random sample. <b>Successes</b> is the number of possible results in the total population. <b>N_population</b> is the size of the total population.                                                                                            |
| INTERCEPT(data_Y; data_X)                         | Calculates the y-value at which a line will intersect the y-axis by using known x-values and y-values. <b>Data_Y</b> is the dependent set of observations or data. <b>Data_X</b> is the independent set of observations or data.<br><br>Names, arrays, or references containing numbers must be used here. Numbers can also be entered directly.                                            |
| KURT(number_1; number_2; ... number_30)           | Returns the kurtosis of a dataset (at least 4 values are required). <b>Number_1; number_2; ... number_30</b> are numerical arguments or ranges representing a random sample of distribution.                                                                                                                                                                                                |
| LARGE(data; rank_c)                               | Returns the Rank_c-th largest value in a data set. <b>Data</b> is the cell range of data. <b>Rank_c</b> is the ranking of the value (2nd largest, 3rd largest, etc.) written as an integer.                                                                                                                                                                                                 |
| LOGINV(number; mean; STDEV)                       | Returns the inverse of the lognormal distribution for the given <b>Number</b> , a probability value. <b>Mean</b> is the arithmetic mean of the standard logarithmic distribution. <b>STDEV</b> is the standard deviation of the standard logarithmic distribution.                                                                                                                          |
| LOGNORMDIST(number; mean; STDEV; cumulative)      | Returns the cumulative lognormal distribution or the probability density for the given <b>Number</b> . <b>Mean</b> is the mean value of the standard logarithmic distribution. <b>STDEV</b> is the standard deviation of the standard logarithmic distribution. <b>Cumulative</b> (optional) = 0 calculates the density function; Cumulative = 1, the default, calculates the distribution. |
| MAX(number_1; number_2; ... number_30)            | Returns the maximum value in a list of arguments. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges.                                                                                                                                                                                                                                                                  |
| MAXA(value_1; value_2; ... value_30)              | Returns the maximum value in a list of arguments. Unlike MAX, text can be entered. The value of the text is 0. <b>Value_1; value_2; ... value_30</b> are values or ranges.                                                                                                                                                                                                                  |

| <b>Syntax</b>                             | <b>Description</b>                                                                                                                                                                                                                                                                          |
|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MEDIAN(number_1; number_2; ... number_30) | Returns the median of a set of numbers. <b>Number_1; number_2; ... number_30</b> are values or ranges, which represent a sample. Each number can also be replaced by a reference.                                                                                                           |
| MIN(number_1; number_2; ... number_30)    | Returns the minimum value in a list of arguments. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges.                                                                                                                                                                  |
| MINA(value_1; value_2; ... value_30)      | Returns the minimum value in a list of arguments. Here text can also be entered. The value of the text is 0. <b>Value_1; value_2; ... value_30</b> are values or ranges.                                                                                                                    |
| MODE(number_1; number_2; ... number_30)   | Returns the most common value in a data set. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges. If several values have the same frequency, it returns the smallest value. An error occurs when a value does not appear twice.                                         |
| NEGBINOMDIST(X; R; SP)                    | Returns the negative binomial distribution. <b>X</b> is the number of unsuccessful tests. <b>R</b> is the number of successful tests. <b>SP</b> is the probability of the success of an attempt.                                                                                            |
| NORMDIST(number; mean; STDEV; C)          | Returns the normal distribution for the given <b>Number</b> in the distribution. <b>Mean</b> is the mean value of the distribution. <b>STDEV</b> is the standard deviation of the distribution. <b>C</b> = 0 calculates the density function, and <b>C</b> = 1 calculates the distribution. |
| NORMINV(number; mean; STDEV)              | Returns the inverse of the normal distribution for the given <b>Number</b> in the distribution. <b>Mean</b> is the mean value in the normal distribution. <b>STDEV</b> is the standard deviation of the normal distribution.                                                                |
| NORMSDIST(number)                         | Returns the standard normal cumulative distribution for the given <b>Number</b> .                                                                                                                                                                                                           |
| NORMSINV(number)                          | Returns the inverse of the standard normal distribution for the given <b>Number</b> , a probability value.                                                                                                                                                                                  |
| PEARSON(data_1; data_2)                   | Returns the Pearson product-moment correlation coefficient r. <b>Data_1</b> is the array of the first data set. <b>Data_2</b> is the array of the second data set.                                                                                                                          |
| PERCENTILE(data; alpha)                   | Returns the alpha-percentile of data values in an array. <b>Data</b> is the array of data. <b>Alpha</b> is the percentage of the scale, between 0 and 1.                                                                                                                                    |

| <b>Syntax</b>                           | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PERCENTRANK(data; value)                | Returns the percentage rank (percentile) of the given <b>value</b> in a sample. <b>Data</b> is the array of data in the sample.                                                                                                                                                                                                                                                                                                                                                   |
| PERMUT(count_1; count_2)                | Returns the number of permutations for a given number of objects. <b>Count_1</b> is the total number of objects. <b>Count_2</b> is the number of objects in each permutation.                                                                                                                                                                                                                                                                                                     |
| PERMUTATIONA(count_1; count_2)          | Returns the number of permutations for a given number of objects (repetition allowed). <b>Count_1</b> is the total number of objects. <b>Count_2</b> is the number of objects in each permutation.                                                                                                                                                                                                                                                                                |
| PHI(number)                             | Returns the values of the distribution function for a standard normal distribution for the given <b>Number</b> .                                                                                                                                                                                                                                                                                                                                                                  |
| POISSON(number; mean; C)                | Returns the Poisson distribution for the given <b>Number</b> . <b>Mean</b> is the mean value of the Poisson distribution. <b>C</b> = 0 calculates the density function, and <b>C</b> = 1 calculates the distribution.                                                                                                                                                                                                                                                             |
| PROB(data; probability; start; end)     | Returns the probability that values in a range are between two limits. Data is the array or range of data in the sample. <b>Probability</b> is the array or range of probabilities of each <b>Data</b> value. <b>Start</b> is the start value of the interval whose probabilities are to be summed. <b>End</b> (optional) is the end value of the interval whose probabilities are to be summed. If this parameter is missing, the probability for the Start value is calculated. |
| QUARTILE(data; type)                    | Returns the quartile of a data set. <b>Data</b> is the array of data in the sample. <b>Type</b> is the type of quartile. (0 = Min, 1 = 25%, 2 = 50% (Median), 3 = 75% and 4 = Max.).                                                                                                                                                                                                                                                                                              |
| RANK(value; data; type)                 | Returns the rank, that is, where the value occurs, of the given <b>Value</b> in a sample. <b>Data</b> is the array or range of data in the sample. <b>Type</b> (optional) is the sequence order, either the default setting of ascending (0) or descending (1).                                                                                                                                                                                                                   |
| RSQ(data_Y; data_X)                     | Returns the square of the Pearson correlation coefficient based on the given values. <b>Data_Y</b> is an array or range of data points. <b>Data_X</b> is an array or range of data points.                                                                                                                                                                                                                                                                                        |
| SKEW(number_1; number_2; ... number_30) | Returns the skewness of a distribution. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges.                                                                                                                                                                                                                                                                                                                                                                  |

| <b>Syntax</b>                            | <b>Description</b>                                                                                                                                                                                                                    |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SLOPE(data_Y; data_X)                    | Returns the slope of the linear regression line. <b>Data_Y</b> is the array or matrix of Y data. <b>Data_X</b> is the array or matrix of X data.                                                                                      |
| SMALL(data; rank_c)                      | Returns the Rank_c-th smallest value in a data set. <b>Data</b> is the cell range of data. <b>Rank_c</b> is the rank of the value (2nd smallest, 3rd smallest, etc.) written as an integer.                                           |
| STANDARDIZE(number; mean; STDEV)         | Converts a random variable to a normalized value. <b>Number</b> is the value to be standardized. <b>Mean</b> is the arithmetic mean of the distribution. <b>STDEV</b> is the standard deviation of the distribution.                  |
| STDEV(number_1; number_2; ... number_30) | Estimates the standard deviation based on a sample. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges representing a sample based on an entire population.                                                      |
| STDEVA(value_1; value_2; ... value_30)   | Calculates the standard deviation of an estimation based on a sample. <b>Value_1; value_2; ... value_30</b> are values or ranges representing a sample derived from an entire population. Text has the value 0.                       |
| STDEV(number_1; number_2; ... number_30) | Calculates the standard deviation based on the entire population. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges representing an entire population.                                                          |
| STDEVPA(value_1; value_2; ! value_30)    | Calculates the standard deviation based on the entire population. <b>Value_1; value_2; ... value_30</b> are values or ranges representing an entire population. Text has the value 0.                                                 |
| STEYX(data_Y; data_X)                    | Returns the standard error of the predicted y value for each x in the regression. <b>Data_Y</b> is the array or matrix of Y data. <b>Data_X</b> is the array or matrix of X data.                                                     |
| TDIST(number; degrees_freedom; mode)     | Returns the t-distribution for the given <b>Number</b> . <b>Degrees_freedom</b> is the number of degrees of freedom for the t-distribution. <b>Mode</b> = 1 returns the one-tailed test, <b>Mode</b> = 2 returns the two-tailed test. |
| TINV(number; degrees_freedom)            | Returns the inverse of the t-distribution for the given <b>Number</b> associated with the two-tailed t-distribution. <b>Degrees_freedom</b> is the number of degrees of freedom for the t-distribution.                               |

| <b>Syntax</b>                           | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TRIMMEAN(data; alpha)                   | Returns the mean of a data set without the Alpha proportion of data at the margins. <b>Data</b> is the array of data in the sample. <b>Alpha</b> is the proportion of the marginal data that will not be considered.                                                                                                                                                                                                                   |
| TTEST(data_1; data_2; mode; type)       | Returns the probability associated with a Student's t-test. <b>Data_1</b> is the dependent array or range of data for the first record. <b>Data_2</b> is the dependent array or range of data for the second record. <b>Mode</b> = 1 calculates the one-tailed test, <b>Mode</b> = 2 the two-tailed test. <b>Type</b> of t-test to perform: paired (1), equal variance (homoscedastic) (2), or unequal variance (heteroscedastic) (3). |
| VAR(number_1; number_2; ... number_30)  | Estimates the variance based on a sample. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges representing a sample based on an entire population.                                                                                                                                                                                                                                                                 |
| VARA(value_1; value_2; ... value_30)    | Estimates a variance based on a sample. The value of text is 0. <b>Value_1; value_2; ... value_30</b> are values or ranges representing a sample based on an entire population.                                                                                                                                                                                                                                                        |
| VARP(Number_1; number_2; ... number_30) | Calculates a variance based on the entire population. <b>Number_1; number_2; ... number_30</b> are numerical values or ranges representing an entire population.                                                                                                                                                                                                                                                                       |
| VARPA(value_1; value_2; ... value_30)   | Calculates the variance based on the entire population. The value of text is 0. <b>Value_1; value_2; ... value_30</b> are values or ranges representing an entire population.                                                                                                                                                                                                                                                          |
| WEIBULL(number; alpha; beta; C)         | Returns the values of the Weibull distribution for the given <b>Number</b> . <b>Alpha</b> is the Alpha (shape) parameter of the Weibull distribution. <b>Beta</b> is the Beta (scale) parameter of the Weibull distribution. <b>C</b> indicates the type of function: C=0, the probability density function is calculated; C=1, the distribution is calculated.                                                                        |
| ZTEST(data; mean; sigma)                | Returns the two-tailed P value of a Z-test for data with a normal distribution. <b>Data</b> is the array of data. <b>Mean</b> is the known mean. <b>Sigma</b> (optional) is the standard deviation of the total population. If this argument is missing, the standard deviation of the sample is processed.                                                                                                                            |

## Date and Time Functions

Use these functions to insert, edit, and manipulate dates and times. OpenOffice handles and computes a date/time value as a number. When you assign the number format "Number" to a date or time value, it is displayed as a number. For example, 01/01/2000 12:00 PM converts to 36526.5. This is simply a matter of formatting; the actual value is always stored and manipulated as a number. To see the date or time displayed in a standard format, change the number format (date or time) accordingly.

Table 5: Data and time functions

| Syntax                        | Description                                                                                                                                                                                                                                                                                                                 |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DATE(year; month; day)        | Converts a date written as year, month, day to an internal serial number and displays it in the cell's formatting. <b>Year</b> is an integer between 1583 and 9956 or 0 and 99. <b>Month</b> is an integer between 1 and 12. <b>Day</b> is an integer between 1 and 31.                                                     |
| DATEVALUE("Text")             | Returns the internal date number for text in quotes. <b>Text</b> is a valid date expression and must be entered with quotation marks.                                                                                                                                                                                       |
| DAY(number)                   | Returns the day, as an integer, of the given date value. A negative date/time value can be entered. <b>Number</b> is a date value.                                                                                                                                                                                          |
| DAYS(date_2; date_1)          | Calculates the difference, in days, between two date values. <b>Date_1</b> is the start date. <b>Date_2</b> is the end date. If <b>Date_2</b> is an earlier date than <b>Date_1</b> , the result is a negative number.                                                                                                      |
| DAYS360(date_1; date_2; type) | Returns the difference between two dates based on the 360-day year used in interest calculations. If <b>Date_2</b> is earlier than <b>Date_1</b> , the function will return a negative number. <b>Type</b> (optional) determines the type of difference calculation: the US method (0) or the European method ( $\neq 0$ ). |
| DAYSINMONTH(date)             | Calculates the number of days in the month of the given <b>date</b> .                                                                                                                                                                                                                                                       |
| DAYSINYEAR(date)              | Calculates the number of days in the year of the given <b>date</b> .                                                                                                                                                                                                                                                        |
| EASTERSUNDAY(integer)         | Returns the date of Easter Sunday for the entered year. <b>Year</b> is an integer between 1583 and 9956 or 0 and 99.                                                                                                                                                                                                        |
| EDATE(start_date; months)     | The result is a date a number of <b>Months</b> away from the given <b>Start_date</b> . <b>Months</b> is the number of months. For example, =EDATE("2020-12-31";2) returns February 28, 2021.                                                                                                                                |
| EOMONTH(start_date; months)   | Returns the date of the last day of a month that falls <b>Months</b> away from the given <b>Start_date</b> . <b>Months</b> is the number of months before (negative) or after (positive) the start date.                                                                                                                    |

| <b>Syntax</b>                               | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HOUR(number)                                | Returns the hour, as an integer, for the given time value. <b>Number</b> is a time value.                                                                                                                                                                                                                                                                                                                                                                    |
| ISLEAPYEAR(date)                            | Determines whether a given <b>date</b> falls within a leap year. Returns either 1 (TRUE) or 0 (FALSE).                                                                                                                                                                                                                                                                                                                                                       |
| MINUTE(number)                              | Returns the minute, as an integer, for the given time value. <b>Number</b> is a time value.                                                                                                                                                                                                                                                                                                                                                                  |
| MONTH(number)                               | Returns the month, as an integer, for the given date value. <b>Number</b> is a time value.                                                                                                                                                                                                                                                                                                                                                                   |
| MONTHS(start_date; end_date; type)          | Calculates the difference, in months, between two date values. <b>Type</b> is one of two possible values, 0 (interval) or 1 (in calendar months). If <b>end_date</b> is an earlier date than <b>start_date</b> , the result is a negative number.                                                                                                                                                                                                            |
| NETWORKDAYS(start_date; end_date; holidays) | Returns the number of workdays between <b>start_date</b> and <b>end_date</b> . Holidays can be deducted. <b>Start_date</b> is the date from which the calculation is carried out. <b>End_date</b> is the date up to which the calculation is carried out. If the start or end date is a workday, the day is included in the calculation. <b>Holidays</b> (optional) is a list of holidays. Enter a cell range in which the holidays are listed individually. |
| NOW()                                       | Returns the computer system date and time. The value is updated when your document recalculates. NOW is a function without arguments.                                                                                                                                                                                                                                                                                                                        |
| SECOND(number)                              | Returns the second, as an integer, for the given time value. <b>Number</b> is a time value.                                                                                                                                                                                                                                                                                                                                                                  |
| TIME(hour; minute; second)                  | Returns a time value from values for hours, minutes, and seconds. This function can be used to convert a time based on these three elements to a decimal time value. <b>Hour</b> , <b>minute</b> , and <b>second</b> must all be integers.                                                                                                                                                                                                                   |
| TIMEVALUE(text)                             | Returns the internal time number from a <b>text</b> enclosed by quotes in a time entry format. The internal number indicated as a decimal is the result of the date system used in OpenOffice to calculate date entries.                                                                                                                                                                                                                                     |
| TODAY()                                     | Returns the current computer system date. The value is updated when your document recalculates. As the empty parentheses indicate, TODAY is a function without arguments.                                                                                                                                                                                                                                                                                    |



| <b>Syntax</b>                         | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| WEEKDAY(number; type)                 | Returns the day of the week for the given <b>number</b> (date value). The day is returned as an integer based on the type. <b>Type</b> determines the type of calculation: type = 1 (default), the weekdays are counted starting from Sunday (Monday = 2); type = 2, the weekdays are counted starting from Monday (Monday = 1); type = 3, the weekdays are counted starting from Monday (Monday = 0). |
| WEEKNUM(number; mode)                 | Calculates the number of the calendar week of the year for the date argument <b>number</b> . <b>Mode</b> sets the start of the week and the calculation type: 1 = Sunday, 2 = Monday.                                                                                                                                                                                                                  |
| WEEKNUM_ADD(date; return_type)        | Calculates the calendar week of the year for a <b>Date</b> . <b>Date</b> is the date within the calendar week. <b>Return_type</b> sets the start of the week and the calculation type: 1 = Sunday, 2 = Monday.                                                                                                                                                                                         |
| WEEKS(start_date; end_date; type)     | Calculates the difference in weeks between two dates, <b>start_date</b> and <b>end_date</b> . <b>Type</b> is one of two possible values, 0 (interval) or 1 (in numbers of weeks).                                                                                                                                                                                                                      |
| WEEKSINYEAR(date)                     | Calculates the number of weeks in a year until a certain <b>date</b> . A week that spans two years is added to the year in which most days of that week occur.                                                                                                                                                                                                                                         |
| WORKDAY(start_date; days; holidays)   | Returns a date number that can be formatted as a date. You then see the date of a day that is a certain number of <b>Workdays</b> away from the <b>start_date</b> . <b>Holidays</b> (optional) is a list of holidays. Enter a cell range in which the holidays are listed individually.                                                                                                                |
| YEAR(number)                          | Returns the value of the year portion of a date. <b>Number</b> shows the date value for which the year is to be returned.                                                                                                                                                                                                                                                                              |
| YEARFRAC(start_date; end_date; basis) | Returns a number between 0 and 1, representing the fraction of a year between <b>start_date</b> and <b>end_date</b> . <b>Start_date</b> and <b>end_date</b> are two date values. <b>Basis</b> is chosen from a list of options and indicates how the year is to be calculated.                                                                                                                         |
| YEARS(start_date; end_date; type)     | Calculates the difference in years between two dates: the <b>start_date</b> and the <b>end_date</b> . <b>Type</b> calculates the type of difference.                                                                                                                                                                                                                                                   |

## Logical Functions

Use the logical functions to test values and produce results based on the result of the test. These functions are conditional and provide the ability to write longer formulas based on input or output.

Table 6: Logical functions

| Syntax                                                      | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| AND(logical_value_1; logical_value_2; ... logical_value_30) | Returns TRUE if all arguments are TRUE. If any element is FALSE, this function returns the FALSE value.<br><b>Logical_value_1; logical_value_2; ... logical_value_30</b> are conditions to be checked. All conditions can be either TRUE or FALSE. If a range is entered as a parameter, the function uses the value from the range that is in the current column or row. The result is TRUE if the logical value in all cells within the cell range is TRUE. |
| FALSE()                                                     | Set the logical value to FALSE. The FALSE() function does not require any arguments.                                                                                                                                                                                                                                                                                                                                                                          |
| IF(test; then_value; otherwise_value)                       | Returns a value depending on a logical test. <b>Test</b> is any value or expression that can be TRUE or FALSE.<br><b>Then_value</b> (optional) is the value that is returned if the logical test is TRUE. <b>Otherwise_value</b> (optional) is the value that is returned if the logical test is FALSE.                                                                                                                                                       |
| NOT(logical_value)                                          | Reverses the logical value. <b>Logical_value</b> is any value to be reversed.                                                                                                                                                                                                                                                                                                                                                                                 |
| OR(logical_value_1; logical_value_2; ... logical_value_30)  | Returns TRUE if at least one argument is TRUE. Returns the value FALSE if all the arguments have the logical value FALSE. <b>Logical_value_1; logical_value_2; ... logical_value_30</b> are conditions to be checked. All conditions can be either TRUE or FALSE. If a range is entered as a parameter, the function uses the value from the range that is in the current column or row.                                                                      |
| TRUE()                                                      | Sets the logical value to TRUE. The TRUE() function does not require any arguments.                                                                                                                                                                                                                                                                                                                                                                           |

## Informational Functions

These functions provide information (or feedback) regarding the results of a test for a specific condition, or a test for the type of data or content a cell contains.

Table 7: Informational functions

| Syntax                     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CELL(info_type; reference) | Returns information on a cell, such as its address, formatting, or contents, based on the value of the <b>info_type</b> argument. <b>Info_type</b> specifies the type of information to be returned and comes from a predefined list of arguments. <b>Info_type</b> is not case sensitive, but it must be enclosed within quotes.<br><b>Reference</b> is the address of the cell to be examined. If <b>reference</b> is a range, the cell reference moves to the top left of the range. If <b>reference</b> is missing, Calc uses the position of the cell in which this formula is located. |

| <b>Syntax</b>        | <b>Description</b>                                                                                                                                                                                                                                                                                                           |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CURRENT()            | Calculates the current value of a formula at the actual position within the formula.                                                                                                                                                                                                                                         |
| FORMULA(reference)   | Displays the formula of a formula cell determined by <b>reference</b> . The formula will be returned as a string. If <b>reference</b> is not a formula cell, or if the presented argument is not a reference, returns the error value #N/A.                                                                                  |
| ISBLANK(value)       | Returns TRUE if the reference to a cell is blank. This function is used to determine if the content of a cell is empty. A cell with a formula inside is not empty. <b>Value</b> is the cell to be tested.                                                                                                                    |
| ISERR(value)         | Returns TRUE if the value refers to any error value except #N/A. You can use this function to control error values in certain cells. If an error occurs, the function returns TRUE. <b>Value</b> is any value or expression in which a test is performed to determine whether an error value, not equal to #N/A, is present. |
| ISERROR(value)       | ISERROR tests if the cells contain general error values. ISERROR recognizes the #N/A error value. If an error occurs, the function returns TRUE. <b>Value</b> is any value where a test is performed to determine whether it is an error value.                                                                              |
| ISEVEN_ADD(number)   | Tests for even numbers. Returns TRUE (1) if the <b>number</b> returns a whole number when divided by 2.                                                                                                                                                                                                                      |
| ISFORMULA(reference) | Returns TRUE if a cell is a formula cell. <b>Reference</b> indicates the reference to a cell to test.                                                                                                                                                                                                                        |
| ISLOGICAL(value)     | Returns TRUE if the cell contains a logical number format. The function is used to check for both TRUE and FALSE values in certain cells. <b>Value</b> is the cell to be tested for logical number format.                                                                                                                   |
| ISNA(value)          | Returns TRUE if a cell contains the #N/A (value not available) error value. <b>Value</b> is the value or expression to be tested.                                                                                                                                                                                            |
| ISNONTTEXT(value)    | Tests if the cell contents are text or numbers, and returns FALSE if the contents are text. <b>Value</b> is any value or expression where a test is performed to determine whether it is a text, or numbers, or a Boolean value.                                                                                             |
| ISNUMBER(value)      | Returns TRUE if the value refers to a number. <b>Value</b> is any expression to be tested to determine whether it is a number or text.                                                                                                                                                                                       |
| ISODD_ADD(number)    | Returns TRUE (1) if the number does not return a whole number when divided by 2. <b>Number</b> is the number to be tested.                                                                                                                                                                                                   |

| <b>Syntax</b> | <b>Description</b>                                                                                                                                                                                  |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ISREF(value)  | Tests if the content of a cell is a reference. <b>Value</b> is the value to be tested, to determine whether it is a reference.                                                                      |
| ISTEXT(value) | Returns TRUE if the cell contents refer to text. <b>Value</b> is a value, a number, a Boolean value, or an error value to be tested.                                                                |
| N(value)      | Returns the numeric value of its argument. Text, FALSE, and #N/A have a value of zero. <b>Value</b> is the parameter to be converted into a number.                                                 |
| NA()          | Returns the error value #N/A.                                                                                                                                                                       |
| TYPE(value)   | Returns the type of value. <b>Value</b> is a specific value for which the data type is determined. The returned values are: 1 = number, 2 = text, 4 = Boolean value, 8 = formula, 16 = error value. |

## Database Functions

This section deals with functions used with data organized as a single row of data for a single record. The *Database* category should not be confused with the Base database component in OpenOffice. A Calc database is simply a range of cells that comprises a block of related data, where each row contains a separate record. There is no connection between a database in OpenOffice Base and the *Database* category in OpenOffice Calc.

The database functions use the following common arguments:

- **Database** is a range of cells that define the database.
- **Database\_field** specifies the column on which the function operates after the search criteria of the first parameter are applied and the data rows are selected. It is not related to the search criteria itself. The number 0 specifies the whole data range. To reference a column by using the column header name, place quotation marks around the header name.
- **Search\_criteria** is a cell range containing the search criteria. Empty cells in the search criteria range will be ignored.

### Note

All of the **search-criteria** arguments for the database functions support regular expressions. For example, "all.\*" can be entered to find the first location of "all" followed by any characters. To search for text that is also a regular expression, precede every character with a \ character. You can switch the automatic evaluation of regular expressions on and off in **Tools > Options > OpenOffice Calc > Calculate**.

Table 8: Database average

| <b>Syntax</b>                                             | <b>Description</b>                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DAVERAGE(database;<br>database_field;<br>search_criteria) | Returns the average of the values of all cells (fields) in all rows (database records) that match the specified <b>search_criteria</b> . The search supports regular expressions.                                                                                                                                                           |
| DCOUNT(database;<br>database_field;<br>search_criteria)   | Counts the number of rows (records) in a database that match the specified <b>search_criteria</b> and contain numerical values. The search supports regular expressions. For the <b>database_field</b> parameter, enter a cell address to specify the column, or enter the number 0 for the entire database. The parameter cannot be empty. |
| DCOUNTA(database;<br>database_field;<br>search_criteria)  | Counts the number of rows (records) in a database that match the specified <b>search_criteria</b> and contain numeric or alphanumeric values. The search supports regular expressions.                                                                                                                                                      |
| DGET(database;<br>database_field;<br>search_criteria)     | Returns the contents of the referenced cell in a database that matches the specified <b>search_criteria</b> . In case of an error, the function returns either #VALUE! for no row found, or Err502 for more than one cell found.                                                                                                            |
| DMAX(database;<br>database_field;<br>search_criteria)     | Returns the maximum content of a cell (field) in a database (all records) that matches the specified <b>search_criteria</b> . The search supports regular expressions.                                                                                                                                                                      |
| DMIN(database;<br>database_field;<br>search_criteria)     | Returns the minimum content of a cell (field) in a database that matches the specified <b>search_criteria</b> . The search supports regular expressions.                                                                                                                                                                                    |
| DPRODUCT(database;<br>database_field;<br>search_criteria) | Multiplies all cells of a data range where the cell contents match the <b>search_criteria</b> . The search supports regular expressions.                                                                                                                                                                                                    |
| DSTDEV(database;<br>database_field;<br>search_criteria)   | Calculates the standard deviation of a population based on a sample, using the numbers in a database column that match the <b>search_criteria</b> . The records are treated as a sample of data.                                                                                                                                            |
| DSTDEVP(database;<br>database_field;<br>search_criteria)  | Calculates the standard deviation of a population based on all cells of a data range which match the <b>search_criteria</b> . The records from the example are treated as the whole population.                                                                                                                                             |
| DSUM(database;<br>database_field;<br>search_criteria)     | Returns the sum of all cells in a database field in all rows (records) that match the specified <b>search_criteria</b> . The search supports regular expressions.                                                                                                                                                                           |

| <b>Syntax</b>                                          | <b>Description</b>                                                                                                                                                                   |
|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DVAR(database;<br>database_field;<br>search_criteria)  | Returns the variance of all cells of a database field in all records that match the specified <b>search_criteria</b> . The records from the example are treated as a sample of data. |
| DVARP(database;<br>database_field;<br>search_criteria) | Calculates the variance of all cell values in a database field in all records that match the specified <b>search_criteria</b> . The records are treated as an entire population.     |

## Array Functions

Table 9: Array functions

| <b>Syntax</b>                                           | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FREQUENCY(data;<br>classes)                             | Counts how many elements of the cell range <b>data</b> are in each of the bins defined by the cell range <b>classes</b> .                                                                                                                                                                                                                                                                                |
| GROWTH(data_Y;<br>data_X; new_data_X;<br>function_type) | Calculates the points of an exponential trend in an array. <b>Data_Y</b> is the Y Data array. <b>Data_X</b> (optional) is the X Data array. <b>New_Data_X</b> (optional) is the X data array, in which the values are recalculated. Function_type is optional. If function_type = 0, functions in the form $y = m^x$ are calculated. Otherwise, $y = b \cdot m^x$ functions are calculated.              |
| LINEST(data_Y; data_X;<br>linear_type; stats)           | Returns the parameters of a linear trend. <b>Data_Y</b> is the Y Data array. <b>Data_X</b> (optional) is the X Data array. <b>Linear_Type</b> (optional): If the line goes through the zero point, then set Linear_Type = 0. <b>Stats</b> (optional): If Stats=0, only the regression coefficients are calculated. Otherwise, other statistics will be seen.                                             |
| LOGEST(data_Y; data_X;<br>function_type; stats)         | Fits the data to an exponential regression curve ( $y=b \cdot m^x$ ). <b>Data_Y</b> is the Y Data array. <b>Data_X</b> (optional) is the X Data array. <b>Function_type</b> (optional): If function_type = 0, functions in the form $y = m^x$ are calculated. Otherwise, $y = b \cdot m^x$ functions are calculated. <b>Stats</b> (optional). If Stats=0, only the regression coefficient is calculated. |
| MDETERM(array)                                          | Returns the determinant of an array. This function returns a value in the current cell; it is not necessary to define a range for the results. <b>Array</b> is a square array.                                                                                                                                                                                                                           |
| MINVERSE(array)                                         | Returns the inverse array. <b>Array</b> is a square array that is to be inverted.                                                                                                                                                                                                                                                                                                                        |
| MMULT(array1; array2)                                   | Calculates the array product of two arrays. The number of columns for array 1 must match the number of rows for array 2. <b>Array1</b> is the first array used in the array product. <b>Array2</b> is the second array.                                                                                                                                                                                  |

| <b>Syntax</b>                                  | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MUNIT(dimensions)                              | Returns the unitary square array of a certain size. The unitary array is a square array where the main diagonal elements equal 1 and all other array elements are equal to 0. <b>Dimensions</b> refers to the size of the array unit.                                                                                                                                                                              |
| SUMPRODUCT(array 1; array 2; ...array 30)      | Multiplies corresponding elements in the given arrays, and returns the sum of those products. <b>Array 1; array 2;...array 30</b> are arrays whose corresponding elements are to be multiplied. At least one array must be part of the argument list. If only one array is given, all array elements are summed.                                                                                                   |
| SUMX2MY2(array_X; array_Y)                     | Returns the sum of the difference of squares of corresponding values in two arrays. <b>Array_X</b> is the first array. <b>Array_Y</b> is the second array whose elements are to be squared.                                                                                                                                                                                                                        |
| SUMX2PY2(array_X; array_Y)                     | Returns the sum of the sum of squares of corresponding values in two arrays. <b>Array_X</b> is the first array whose arguments are to be squared and added. <b>Array_Y</b> is the second array, whose elements are to be added and squared.                                                                                                                                                                        |
| SUMXMY2(array_X; array_Y)                      | Adds the squares of the difference between corresponding values in two arrays. <b>Array_X</b> is the first array, and <b>Array_Y</b> is the second array.                                                                                                                                                                                                                                                          |
| TRANSPOSE(array)                               | Transposes the rows and columns of an array. <b>Array</b> is the array in the spreadsheet that is to be transposed.                                                                                                                                                                                                                                                                                                |
| TREND(data_Y; data_X; new_data_X; linear_Type) | Returns values along a linear trend. <b>Data_Y</b> is the y-data array. <b>Data_X</b> (optional) is the x-data array. <b>New_data_X</b> (optional) is the array of the x-data, which is used for recalculating values. <b>Linear_type</b> is optional. If linear_type = 0, then lines will be calculated through the zero point. Otherwise, offset lines will also be calculated. The default is linear_type <> 0. |

## Spreadsheet Functions

Use spreadsheet functions to search and address cell ranges and provide feedback regarding the contents of a cell or range of cells. You can use functions such as HYPERLINK() and DDE() to connect to other documents or data sources.

Table 10: Spreadsheet functions

| Syntax                             | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ADDRESS(row; column; abs; sheet)   | Returns a cell address (reference) as text, according to the specified row and column numbers. Optionally, whether the address is interpreted as an absolute address (for example, \$A\$1), a relative address (as A1), or in a mixed form (A\$1 or \$A1) can be determined. The name of the sheet can also be specified. <b>Row</b> is the row number for the cell reference. <b>Column</b> is the column number for the cell reference (the number, not the letter). <b>Abs</b> determines the type of reference. <b>Sheet</b> is the name of the sheet.                                                                                                                                                    |
| AREAS(reference)                   | Returns the number of individual ranges that belong to a multiple range. A range can consist of contiguous cells or a single cell. <b>Reference</b> is the reference to a cell or cell range.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| CHOOSE(index; value1; ... value30) | Uses an index to return a value from a list of up to 30 values. <b>Index</b> is a reference or number between 1 and 30, indicating which value is to be taken from the list. <b>Value1; ... value30</b> is the list of values entered as a reference to a cell or as individual values.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| COLUMN(reference)                  | Returns the column number of a cell reference. If the reference is a cell, the column number of the cell is returned; if the parameter is a cell area, the corresponding column numbers are returned in a single-row array if the formula is entered as an array formula. If the COLUMN function with an area reference parameter is not used for an array formula, only the column number of the first cell within the area is determined. <b>Reference</b> is the reference to a cell or cell area whose first column number is to be found. If no reference is entered, the column number of the cell in which the formula is entered is found. Calc automatically sets the reference to the current cell. |
| COLUMNS(array)                     | Returns the number of columns in the given reference. <b>Array</b> is the reference to a cell range whose total number of columns is to be found. The argument can also be a single cell.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |



| <b>Syntax</b>                                  | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DDE(server; file; range; mode)                 | Returns the result of a DDE-based link. If the contents of the linked range or section change, the returned value will also change. The spreadsheet can be reloaded, or <b>Edit &gt; Links</b> selected, to view the updated links. Cross-platform links, for example, from an OpenOffice installation running on a Windows machine to a document created on a Linux machine, are not supported. <b>Server</b> is the name of a server application. OpenOffice applications have the server name "Soffice". <b>File</b> is the complete file name, including path. <b>Range</b> is the area containing the data to be evaluated. <b>Mode</b> is an optional parameter that controls the method by which the DDE server converts its data into numbers. |
| ERRORTYPE(reference)                           | Returns the number corresponding to an error value occurring in a different cell. With the aid of this number, an error message text can be generated. If an error occurs, the function returns a logical or numerical value. <b>Reference</b> contains the address of the cell in which the error occurs.                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| HLOOKUP(search_criteria; array; index; sorted) | Searches for a value in the first row of a range and returns a value from that column. <b>Search_criteria</b> determines what is searched for. <b>Array</b> sets which cell range is searched. <b>Index</b> sets the row from which a value is returned, relative to the first row of the array. <b>Sorted</b> is TRUE or FALSE, indicating whether the first row is sorted in ascending order.                                                                                                                                                                                                                                                                                                                                                        |
| HYPERLINK(URL) or<br>HYPERLINK(URL; cell_text) | When a cell that contains the HYPERLINK function is clicked, the hyperlink opens. <b>URL</b> specifies the link target. The optional <b>cell_text</b> argument is the text displayed in the cell. If the cell_text parameter is not specified, the <b>URL</b> is displayed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| INDEX(reference; row; column; range)           | Returns the content of a cell, specified by row and column number or an optional range name. <b>Reference</b> is a cell reference, entered either directly or by specifying a range name. If the reference consists of multiple ranges, the reference or range name must be enclosed in parentheses. <b>Row</b> (optional) is the row number of the reference range from which to return a value. <b>Column</b> (optional) is the column number of the reference range from which to return a value. <b>Range</b> (optional) is the index of the subrange if referring to a multiple range.                                                                                                                                                            |

| <b>Syntax</b>                                          | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| INDIRECT(reference)                                    | Returns the <b>reference</b> specified by a text string. This function can also be used to return the area of a corresponding string. <b>Reference</b> is a reference to a cell or an area (in text form) from which to return the contents.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| LOOKUP(search_criterion; search_vector; result_vector) | Returns the contents of a cell either from a one-row or one-column range or from an array. Optionally, the assigned value (of the same index) is returned in a different column and row. Unlike VLOOKUP and HLOOKUP, search and result vectors may be located at different positions; they do not have to be adjacent. Additionally, the search vector for the LOOKUP must be sorted; otherwise, the search will not return any usable results. The search supports regular expressions. <b>Search_criterion</b> is the value to be searched for; entered either directly or as a reference. <b>Search_vector</b> is the single row or column to be searched. <b>Result_vector</b> is another single-row or single-column range from which the result of the function is taken. The result is the cell of the result vector with the same index as the instance found in the search vector. |
| MATCH(search_criterion; lookup_array; type)            | Returns the relative position of an item in an array that matches a specified value. The function returns the position of the value found in the lookup_array as a number. <b>Search_criterion</b> is the value to be searched for in the single-row or single-column array. <b>Lookup_array</b> is the reference searched. <b>Type</b> may take the values 1, 0, or -1 for matching as <=, exact match, or >=. The search supports regular expressions.                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| OFFSET(reference; rows; columns; height; width)        | Returns the value of a cell offset by a certain number of rows and columns from a given reference point. <b>Reference</b> is the cell from which the function searches for the new reference. <b>Rows</b> is the number of rows by which the reference is moved up (negative value) or down. <b>Columns</b> is the number of columns by which the reference is moved to the left (negative value) or to the right. <b>Height</b> is the optional vertical height for an area that starts at the new reference position. <b>Width</b> is the optional horizontal width for an area that starts at the new reference position.                                                                                                                                                                                                                                                                |

| <b>Syntax</b>              | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ROW(reference)             | Returns the row number of a cell reference. If the reference is a cell, it returns the row number of the cell. If the reference is a cell range, it returns the corresponding row numbers in a one-column Array if the formula is entered as an array formula. If the ROW function with a range reference is not used in an array formula, only the row number of the first range cell will be returned. <b>Reference</b> is a cell, an area, or the name of an area. If a reference is not indicated, Calc automatically sets the reference to the current cell. |
| ROWS(array)                | Returns the number of rows in a reference or array. <b>Array</b> is the reference or named area whose total number of rows is to be determined.                                                                                                                                                                                                                                                                                                                                                                                                                   |
| SHEET(reference)           | Returns the sheet number of a reference or a string representing a sheet name. If no parameters are entered, the result is the sheet number of the spreadsheet containing the formula. <b>Reference</b> (optional) is the reference to a cell, an area, or a sheet name string.                                                                                                                                                                                                                                                                                   |
| SHEETS(reference)          | Determines the number of sheets in a reference. If no parameters are entered, the result is the number of sheets in the current document. <b>Reference</b> (optional) is the reference to a sheet or an area.                                                                                                                                                                                                                                                                                                                                                     |
| STYLE(style; time; style2) | Applies a style to the cell containing the formula. After a set amount of time, another style can be applied. This function always returns the value 0, allowing it to be added to another function without changing the value. <b>Style</b> is the name of a cell style assigned to the cell. <b>Time</b> is an optional time in seconds. <b>Style2</b> is the optional name of a cell style assigned to the cell after a certain amount of time has passed.                                                                                                     |

| <b>Syntax</b>                                       | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VLOOKUP(search_criterion; array; index; sort_order) | Searches vertically with reference to adjacent cells to the right. If a specific value is contained in the first column of an array, it returns the value to the same line of a specific array column named by <b>index</b> . The search supports regular expressions. <b>Search_criterion</b> is the value searched for in the first column of the array. <b>Array</b> is the reference, which must include at least two columns. <b>Index</b> is the number of the column in the array that contains the value to be returned. The first column has the number 1. <b>Sort_order</b> (optional) indicates whether the first column in the array is sorted in ascending order. If the first column is not sorted, <b>Sort_order</b> must be set to zero. |

## Text Functions

Use Calc's text functions to search and manipulate text strings or character codes.

Table 11: Text functions

| <b>Syntax</b>                         | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ARABIC(text)                          | Calculates the value of a Roman number. The value range must be between 0 and 3999. <b>Text</b> is the text that represents a Roman number.                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| BASE(number; radix; [minimum_length]) | Converts a positive integer to a specified base, then into text using the characters from the base's numbering system (decimal, binary, hexadecimal, etc.). Only the digits 0-9 and the letters A-Z are used. <b>Number</b> is the positive integer to be converted. <b>Radix</b> is the base of the number system. It may be any positive integer between 2 and 36. <b>Minimum_length</b> (optional) is the minimum length of the character sequence that has been created. If the text is shorter than the indicated minimum length, zeros are added to the left of the string. |
| CHAR(number)                          | Converts a number into a character according to the current code table. The number can be a two-digit or three-digit integer number. <b>Number</b> is a number between 1 and 255 representing the code value for the character.                                                                                                                                                                                                                                                                                                                                                   |
| CLEAN(text)                           | Removes all non-printing characters from the string. <b>Text</b> refers to the text from which to remove all non-printable characters.                                                                                                                                                                                                                                                                                                                                                                                                                                            |

| <b>Syntax</b>                                   | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CODE(text)                                      | Returns a numeric code for the first character in a text string. <b>Text</b> is the text for which the code of the first character is to be found.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| CONCATENATE(text_1; text_2; ... text_30)        | Combines several text strings into one string. <b>Text_1; text_2; ... text_30</b> are text passages that are to be combined into one string.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| DECIMAL(text; radix)                            | Converts text with characters from a number in the base radix given to a positive integer in base 10. The radix must be in the range 2 to 36. Spaces and tabs are ignored. The <b>text</b> field is not case-sensitive. <b>Text</b> is the text to be converted. To differentiate between a hexadecimal number, such as A1, and the reference to cell A1, place the number in quotation marks; for example, "A1" or "FACE". <b>Radix</b> is the base of the number system.                                                                                                                                                                   |
| DOLLAR(value; decimals)                         | Converts a number to an amount in the currency format, rounded to a specified decimal place. <b>Value</b> is the number to be converted to currency; it can be a number, a reference to a cell containing a number, or a formula that returns a number. <b>Decimals</b> (optional) is the number of decimal places. If no decimal value is specified, all numbers in currency format will be displayed with two decimal places. The currency format is set in the system settings.                                                                                                                                                           |
| EXACT(text_1; text_2)                           | Compares two text strings and returns TRUE if they are identical. This function is case-sensitive. <b>Text_1</b> is the first text to compare. <b>Text_2</b> is the second text to compare.                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| FIND(find_text; text; position)                 | Looks for a string of text within another string. Where to begin the search can also be defined. The search term can be a number or any string of characters. The search is case-sensitive. <b>Find_text</b> is the text to be found. <b>Text</b> is the text where the search takes place. <b>Position</b> (optional) is the position in the text from which the search starts.                                                                                                                                                                                                                                                             |
| FIXED(number; decimals; no_thousands_separator) | Specifies that a number be displayed with a fixed number of decimal places and with or without a thousands separator. This function can be used to apply a uniform format to a column of numbers. <b>Number</b> is the number to be formatted. <b>Decimals</b> is the number of decimal places to be displayed. <b>No_thousands_separator</b> (optional) determines whether the thousands separator is used or not. If the parameter is a number not equal to 0, the thousands separator is suppressed. If the parameter is equal to 0 or if it is missing altogether, the thousands separators of the current locale setting are displayed. |

| <b>Syntax</b>                             | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LEFT(text; number)                        | Returns the first character or characters in a text string. <b>Text</b> is the text from which the initial partial words are to be determined. <b>Number</b> (optional) is the number of characters to be returned from the start text. If this parameter is not defined, one character is returned.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| LEN(text)                                 | Returns the length of a string, including spaces. <b>Text</b> is the text whose length is to be determined.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| LOWER(text)                               | Converts all uppercase letters in a text string to lowercase. <b>Text</b> is the text to be converted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| MID(text; start; number)                  | Returns a text segment of a character string. The parameters specify the starting position and the number of characters. <b>Text</b> is the text containing the characters to extract. <b>Start</b> is the position of the first character in the text to extract. <b>Number</b> is the number of characters in the part of the text.                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| PROPER(text)                              | Capitalizes the first letter in all words of a text string. <b>Text</b> is the text to be converted.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| REPLACE(text; position; length; new_text) | Replaces part of a text string with a different text string. This function can be used to replace both characters and numbers (which are automatically converted to text). The result of the function is always displayed as text. To perform further calculations with a number that has been replaced by text, convert it back to a number using the VALUE function. Any text containing numbers must be enclosed in quotation marks so it is not interpreted as a number and automatically converted to text. <b>Text</b> is text of which a part will be replaced. <b>Position</b> is the position within the text where the replacement will begin. <b>Length</b> is the number of characters in text to be replaced. <b>New_text</b> is the text that replaces the selected characters. |
| REPT(text; number)                        | Repeats a character string by the given number of copies. <b>Text</b> is the text to be repeated. <b>Number</b> is the number of repetitions. The result can be a maximum of 255 characters.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| RIGHT(text; number)                       | Extracts the last character or characters in a text string. <b>Text</b> is the text of which the right part is to be determined. <b>Number</b> (optional) is the number of characters from the right part of the text.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

| <b>Syntax</b>                                       | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ROMAN(number; mode)                                 | Converts a number into a Roman numeral. The value range must be between 0 and 3999; the modes can be integers from 0 to 4. <b>Number</b> is the number that is to be converted into a Roman numeral. <b>Mode</b> (optional) indicates the degree of simplification. The higher the value, the greater the simplification of the Roman numeral.                                                                                                               |
| ROT13(text)                                         | Encrypts a character string by moving the characters 13 positions in the alphabet. After the letter Z, the alphabet begins again (Rotation). Applying the encryption function again to the resulting code decrypts the text. <b>Text</b> : Enter the character string to be encrypted. ROT13(ROT13(Text)) decrypts the code.                                                                                                                                 |
| SEARCH(find_text; text; position)                   | Returns the position of a text segment within a character string. The start of the search can be set as an option. The search text can be a number or any sequence of characters. The search is not case-sensitive. The search supports regular expressions. <b>Find_text</b> is the text to be searched for. <b>Text</b> is the text where the search will take place. <b>Position</b> (optional) is the position in the text where the search is to start. |
| SUBSTITUTE(text; search_text; new text; occurrence) | Substitutes new text for old text in a string. <b>Text</b> is the text in which text segments are to be exchanged. <b>Search_text</b> is the text segment that is to be replaced (a number of times). <b>New text</b> is the text that replaces the existing text segment. <b>Occurrence</b> (optional) indicates how many occurrences of the search text are to be replaced. If this parameter is missing, the search text is replaced throughout.          |
| T(value)                                            | Converts a number to a blank text string. <b>Value</b> is the value to be converted. Also, a reference can be used as a parameter. If the referenced cell includes a number or a formula containing a numerical result, the result will be an empty string. If <b>value</b> is text, it is returned unchanged.                                                                                                                                               |
| TEXT(number; format)                                | Converts a number into text according to a given format. <b>Number</b> is the numerical value to be converted. <b>Format</b> is the text that defines the format. Use decimal and thousands separators according to the language set in the cell format.                                                                                                                                                                                                     |
| TRIM(text)                                          | Removes leading and trailing spaces, or multiple internal spaces from a string. <b>Text</b> is the text in which spaces are removed.                                                                                                                                                                                                                                                                                                                         |
| UPPER(text)                                         | Converts the string specified in the <b>text</b> parameter to uppercase.                                                                                                                                                                                                                                                                                                                                                                                     |

| <b>Syntax</b> | <b>Description</b>                                                                         |
|---------------|--------------------------------------------------------------------------------------------|
| VALUE(text)   | Converts a text string into a number. <b>Text</b> is the text to be converted to a number. |

## Converting Decimal, Binary, Octal, Hexadecimal

Table 12: Converting Decimal, Binary, Octal, Hexadecimal

| <b>Syntax</b>           | <b>Description</b>                                                                                                                                                      |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BIN2DEC(number)         | Returns the decimal number for the binary number entered. <b>Number</b> is the binary number.                                                                           |
| BIN2HEX(number; places) | Returns the hexadecimal number for the binary number entered. <b>Number</b> is the binary number. <b>Places</b> is the number of places to be output.                   |
| BIN2OCT(number; places) | Returns the octal number for the binary number entered. <b>Number</b> is the binary number. <b>Places</b> is the number of places to be output.                         |
| DEC2BIN(number; places) | Returns the binary number for the decimal number entered between -512 and 511. <b>Number</b> is the decimal number. <b>Places</b> is the number of places to be output. |
| DEC2HEX(number; places) | Returns the hexadecimal number for the decimal number entered. <b>Number</b> is the decimal number. <b>Places</b> is the number of places to be output.                 |
| DEC2OCT(number; places) | Returns the octal number for the decimal number entered. <b>Number</b> is the decimal number. <b>Places</b> is the number of places to be output.                       |
| HEX2BIN(number; places) | Returns the binary number for the hexadecimal number entered as a string. <b>Number</b> is the hexadecimal number. <b>Places</b> is the number of places to be output.  |
| HEX2DEC(number)         | Returns the decimal number for the hexadecimal number entered as a string. <b>Number</b> is the hexadecimal number.                                                     |
| HEX2OCT(number; places) | Returns the octal number for the hexadecimal number entered as a string. <b>Number</b> is the hexadecimal number. <b>Places</b> is the number of places to be output.   |
| OCT2BIN(number; places) | Returns the binary number for the octal number entered. <b>Number</b> is the octal number. <b>Places</b> is the number of places to be output.                          |
| OCT2DEC(number)         | Returns the decimal number for the octal number entered. <b>Number</b> is the octal number.                                                                             |



| <b>Syntax</b>           | <b>Description</b>                                                                                                                                  |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| OCT2HEX(number; places) | Returns the hexadecimal number for the octal number entered. <b>Number</b> is the octal number. <b>Places</b> is the number of places to be output. |

## Complex Number Functions

Table 13 Complex number functions

| <b>Syntax</b>                    | <b>Description</b>                                                                                                                                                                                                                                             |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| COMPLEX(real_num; i_num; suffix) | Returns a complex number from a real coefficient and an imaginary coefficient. <b>Real_num</b> is the real coefficient of the complex number. <b>I_num</b> is the imaginary coefficient of the complex number. <b>Suffix</b> is a list of options, "i" or "j". |
| IMABS(complex_number)            | Returns the absolute value (modulus) of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                                                                                                                              |
| IMAGINARY(complex_number)        | Returns the imaginary coefficient of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                                                                                                                                 |
| IMARGUMENT(complex_number)       | Returns the argument (the phi angle in radians) of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                                                                                                                   |
| IMCONJUGATE(complex_number)      | Returns the conjugated complex complement to a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                                                                                                                         |
| IMCOS(complex_number)            | Returns the cosine of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                                                                                                                                                |
| IMDIV(numerator; denominator)    | Returns the division of two complex numbers. <b>Numerator</b> , <b>Denominator</b> are entered in the form "x + yi" or "x + yj".                                                                                                                               |
| IMEXP(complex_number)            | Returns the power of e (the Eulerian number) and the <b>complex_number</b> . The complex_number is entered in the form "x + yi" or "x + yj".                                                                                                                   |
| IMLN(complex_number)             | Returns the natural logarithm of a <b>complex_number</b> . The complex_number is entered in the form "x + yi" or "x + yj".                                                                                                                                     |
| IMLOG10(complex_number)          | Returns the common logarithm of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                                                                                                                                      |

| <b>Syntax</b>                                    | <b>Description</b>                                                                                                                                    |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| IMLOG2(complex_number)                           | Returns the binary logarithm of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                             |
| IMPOWER(complex_number; number)                  | Returns the integer power of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj". <b>Number</b> is the exponent. |
| IMPRODUCT(complex_number; complex_number_1; ...) | Returns the product of up to 29 <b>complex_numbers</b> . The complex numbers are entered in the form "x + yi" or "x + yj".                            |
| IMREAL(complex_number)                           | Returns the real coefficient of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                             |
| IMSIN(complex_number)                            | Returns the sine of a <b>complex_number</b> . The complex number is entered in the form "x + yi" or "x + yj".                                         |
| IMSQRT(complex_number)                           | Returns the square root of a <b>complex_number</b> . The complex numbers are entered in the form "x + yi" or "x + yj".                                |
| IMSUB(complex_number_1; complex_number_2)        | Returns the subtraction of two <b>complex_numbers</b> . The <b>complex_numbers</b> are entered in the form "x + yi" or "x + yj".                      |
| IMSUM(complex_number; complex_number_1; ...)     | Returns the sum of up to 29 complex numbers. The <b>complex_numbers</b> are entered in the form "x + yi" or "x + yj".                                 |